

Proyecto Estratégico “Madurez, Adaptabilidad, Resiliencia y Continuidad en Organizaciones Digitalizadas. MARC.0 Diligencia)” Informe de seguimiento

(Periodo octubre 2024 – febrero 2026)

Índice de contenido

Índice de contenido.....	2
Resumen Ejecutivo	6
Ámbito Institucional	6
Introducción	6
Objetivos.....	7
Objetivos técnicos	8
Investigación.....	10
OpenUI (GitHub)	10
Towards an AI co-scientis	10
Construcción de un grafo de conocimiento con LlamaCloud y Neo4J	11
Contexto y motivación	11
Flujo de trabajo (pipeline).....	11
Aplicación y beneficios	12
LangChain y LlamaIndex	12
N8N	13
MCP	14
A2A y MoE	14
Formación.....	15
Metodología y gestión del proyecto	15
Diseño de la arquitectura	16
Subsistema de Persistencia de Datos	17
Modelado de Grafos en Neo4j	17
Taxonomía de Clasificación.....	18
Extracción de datos	19
Descripción de los scrapers	19
Scraper de Instituto Nacional de Ciberseguridad	19
Scraper de Centro Criptológico Nacional	19
Scraper de National Cyber Security Centre	20
Scraper de World Economic Forum	20

Problemática y Solución en Web Forum (World Economic Forum)	20
Scraper de Canadian Centre for Cyber Security	20
Scraper de National Institute of Standards and Technology	20
Scraper de Cybersecurity and Infrastructure Security Agency	21
Arquitectura funcional	21
1. Módulo de rastreo y extracción de contenido	21
2. Módulo de exploración de secciones	21
Ejecución y configuración.....	21
Escalabilidad y adaptabilidad	21
Normalización y Preprocesamiento.....	22
Arquitectura RAG (Retrieval-Augmented Generation).....	22
Jerarquía de Agentes	23
Clase Base: BaseAgent.....	23
Clase Intermedia: ConversationalAgentBase	24
ConversationalAgent	24
Prompt de sistema:	25
NewsAgent (Agente RAG).....	25
Herramientas disponibles (Tools):.....	26
Flujo de procesamiento (process_query):	27
Prompt de sistema del NewsAgent:	27
RouterAgent (Orquestador)	27
Prompt de sistema del RouterAgent:.....	28
Criterios de routing:	29
Proceso general	29
Estrategia de Vectorización (Embeddings).....	30
Interfaz API REST	30
Organización modular	31
Sistema de autenticación	34
Sesiones asíncronas	36
Redis	37
Middlewares y mecanismos de seguridad de la API	38

Despliegue	40
Frontend	42
API_CLI	42
Alternativa web propia	45
Alternativa web en Wordpress	50
Despliegue en SCAYLE	50
Resultados obtenidos	52
Análisis de problemas y gestión de riesgos	52
Retraso en la Infraestructura (SCAYLE)	52
Obsolescencia de los Scrapers (Caso Web Forum)	52
Cambios en el Equipo de Desarrollo	52
Conclusiones	52
Líneas de trabajo futuras	53
Anexo	54
Especificación de la API REST	54
Autenticación (Auth)	54
Gestión de Usuarios (Users)	56
Conversaciones / Chats	57
Plantillas de IA (Templates)	60
Modelos de IA (Models)	61
Sistema (System)	62
API V2 (Endpoints Equivalentes)	62
Autenticación V2	62
Gestión de Usuarios V2	64
Conversaciones / Chats V2	65
Participantes	68

Figura 1 Pantalla principal de OpenUI.	10
Figura 2 Logo de llama-index.	11
Figura 3 Logo de neo4j.	11
Figura 4 Principales tecnologías empleadas.	11
Figura 5 Esquema general de la arquitectura del proyecto.	17

Resumen Ejecutivo

El presente informe técnico detalla los resultados finales del proyecto MARC.0 (Madurez, Adaptabilidad, Resiliencia y Continuidad en Organizaciones Digitalizadas).

El propósito fundamental de este desarrollo ha sido la creación de una plataforma soberana de inteligencia en ciberseguridad capaz de ingerir, procesar y relacionar grandes volúmenes de información no estructurada con la que servir un modelo de IA conversacional (chatbot). A fecha de cierre de este informe, el sistema cuenta con una base de conocimiento de más de 30.000 artículos técnicos y noticias de ciberseguridad, estructurados en un grafo de conocimiento que permite consultas complejas y razonamiento asistido por Inteligencia Artificial.

La ejecución del proyecto ha requerido un notable esfuerzo técnico para la adquisición de datos y en la orquestación de modelos de lenguaje masivos (LLMs). Asimismo, el despliegue final se ha realizado en el Centro de Supercomputación de Castilla y León (SCAYLE).

Ámbito Institucional

Este proyecto se ha ejecutado en la Universidad de León (ULE) con la financiación del Instituto Nacional de Ciberseguridad (INCIBE) a través de fondos europeos.

Introducción

En un entorno cada vez más digitalizado, la ciberseguridad se ha convertido en una prioridad para gobiernos, empresas y ciudadanos. El constante aumento de amenazas digitales y la creciente sofisticación de los ciberataques requieren soluciones tecnológicas capaces de prevenir, detectar y responder de manera eficiente. En este contexto, surge el proyecto MARCO Digiliencia, una iniciativa desarrollada por estudiantes de la Universidad de León, con el objetivo de ofrecer una herramienta moderna, escalable y segura que fortalezca la protección digital de los ciudadanos.

El proyecto está enfocado en dar soporte al servicio 017, una línea de ayuda en ciberseguridad que atiende incidencias, proporciona formación y ofrece retroalimentación a los usuarios. Dado el volumen de consultas y la necesidad de atención especializada en distintos niveles (N1, N2 y N3), se propone la creación de un chatbot de ciberseguridad, potenciado por inteligencia artificial y respaldado por una base de datos orientada a grafos. Esta solución busca facilitar el acceso a información confiable, ofrecer respuestas automáticas a problemas comunes y derivar casos complejos a agentes humanos cuando sea necesario.

La inteligencia artificial, entrenada, validada y optimizada mediante técnicas como el ajuste de hiperparámetros y validación cruzada, permite lograr una alta precisión en la detección de incidencias y la recomendación de soluciones. Este sistema no solo está pensado para actuar ante incidentes ya ocurridos, sino también para prevenir ataques, educar al usuario y fomentar buenas prácticas en ciberseguridad.

En un momento en que los ciberataques representan ya el 1.5% del PIB mundial, herramientas como la que plantea este proyecto son fundamentales para reducir el impacto económico y social de estas amenazas. La finalidad es clara: proteger, informar y capacitar al ciudadano mediante el uso responsable e innovador de la tecnología.

Objetivos

El proyecto tiene como objetivo principal desarrollar un servicio accesible y adaptable a las características de sus usuarios, considerando tanto su nivel técnico como su contexto social, en línea con los principios del diseño centrado en el usuario y la accesibilidad universal. Asimismo, se busca crear una solución escalable, sostenible y de bajo coste, capaz de evolucionar conforme a las necesidades futuras y de garantizar su mantenimiento a largo plazo sin comprometer la calidad del servicio, siguiendo las recomendaciones sobre sostenibilidad tecnológica y eficiencia de recursos. Asimismo, se pretende garantizar que la información proporcionada sea actual, fiable y basada en fuentes verificadas, asegurando la trazabilidad de las respuestas y la transparencia en los procesos de gestión de la información, de acuerdo con los principios de calidad de la información y gobernanza de datos. El proyecto también contempla el cumplimiento de principios éticos, de seguridad y de la normativa vigente, especialmente en lo referente a la protección de datos y la privacidad de los usuarios, en consonancia con los marcos regulatorios y de buenas prácticas establecidos por el Reglamento General de Protección de Datos y las directrices sobre ética y seguridad en sistemas digitales propuestas por organismos internacionales.

Otro de los objetivos del proyecto es facilitar la interacción entre los usuarios y el sistema mediante el diseño e implementación de diferentes interfaces intuitivas, coherentes y fáciles de usar, acompañadas de una documentación clara y accesible. Además, se contempló la implementación de un sistema de gestión de usuarios que permita diferenciar perfiles según sus necesidades, roles y nivel de acceso, garantizando una utilización adecuada de las funcionalidades disponibles.

Finalmente, se busca ofrecer a cada usuario acceso a su historial de uso y actividad, permitiendo la consulta y seguimiento de interacciones previas, lo que contribuye a una experiencia más personalizada, continua y orientada a la mejora del servicio y la satisfacción del usuario.

Objetivos técnicos

Desde un punto de vista técnico, el proyecto busca diseñar, implementar y desplegar un servicio de IA conversacional junto a servicios auxiliares de *Web Scraping*, almacenamiento en Grafos de Conocimiento (*Knowledge Graphs*) y Modelos de Lenguaje Grande (LLMs).

Objetivos específicos de ingeniería de datos:

- **Desarrollo de un sistema de ingesta masiva:** implementar una batería de scrapers modulares capaces de extraer información de fuentes heterogéneas. Se estableció como métrica de éxito la capacidad de procesar y normalizar miles de documentos históricos y mantener un flujo diario de actualizaciones.
- **Modelado semántico en grafos:** superar las limitaciones de las bases de datos relacionales mediante la implementación de Neo4j. El objetivo era modelar entidades (amenazas, actores, vulnerabilidades, sectores) y sus relaciones transversales.
- **Superación de barreras anti-automatización:** desarrollar mecanismos robustos para la navegación simulada (vía Selenium y gestión de cabeceras) que permitieran la extracción de datos en portales con alta carga de JavaScript dinámico y medidas anti-bot, asegurando la continuidad del flujo de datos.

Objetivos de inteligencia artificial y procesamiento:

- **Implementación de RAG (Retrieval-Augmented Generation):** integrar un motor de IA que no solo genere texto, sino que fundamente sus respuestas en la base de datos de grafos construida, reduciendo al mínimo las "alucinaciones" típicas de los modelos generativos.
- **Despliegue de modelos en local:** ejecutar modelos de *embeddings* y generación de texto en infraestructura propia para garantizar la privacidad y soberanía del dato, evitando el envío de información sensible a APIs de terceros (como OpenAI o Anthropic).
- **Clasificación automática:** utilizar IA para etiquetar taxonómicamente los más de 30.000 artículos ingestados, permitiendo su categorización por tipo de amenaza, sector afectado e impacto potencial sin intervención humana.

Objetivos de infraestructura y despliegue:

- **Despliegue en SCAYLE:** migrar el desarrollo desde entornos locales a una infraestructura de supercomputación profesional donde emplear modelos de lenguaje de gran tamaño que, por sus prestaciones, son imposibles de ejecutar en máquinas personales.
- **Arquitectura híbrida (virtual/física):** configurar un entorno complejo que combina:

- **Máquinas virtuales:** dedicadas a la persistencia de datos (Neo4j, PostgreSQL), el servidor de backend (API).
- **Máquina física con aceleración GPU:** configuración y puesta en marcha de un servidor con capacidad de ejecución de los modelos de inferencia y vectorización masiva (GPU).
- **Alta disponibilidad y seguridad:** asegurar las comunicaciones mediante cifrado SSL/TLS (gestión de certificados propios) y contenedorización de todos los servicios mediante Docker para facilitar la mantenibilidad y escalabilidad.

Investigación

En la primera fase, se realizó un proceso de investigación para identificar las tecnologías más adecuadas y comprender su propósito dentro del sistema. Paralelamente, se llevaron a cabo formaciones en diversas herramientas con el objetivo de adquirir un conocimiento técnico básico que facilitara su posterior integración.

OpenUI (GitHub)

Esta herramienta fue explorada como posible interfaz de usuario modular y gratuita para la interacción con sistemas complejos. Aunque se valoró su potencial para ofrecer un frontend abierto y reutilizable, quedó solo como propuesta y no fue incorporada al proyecto.

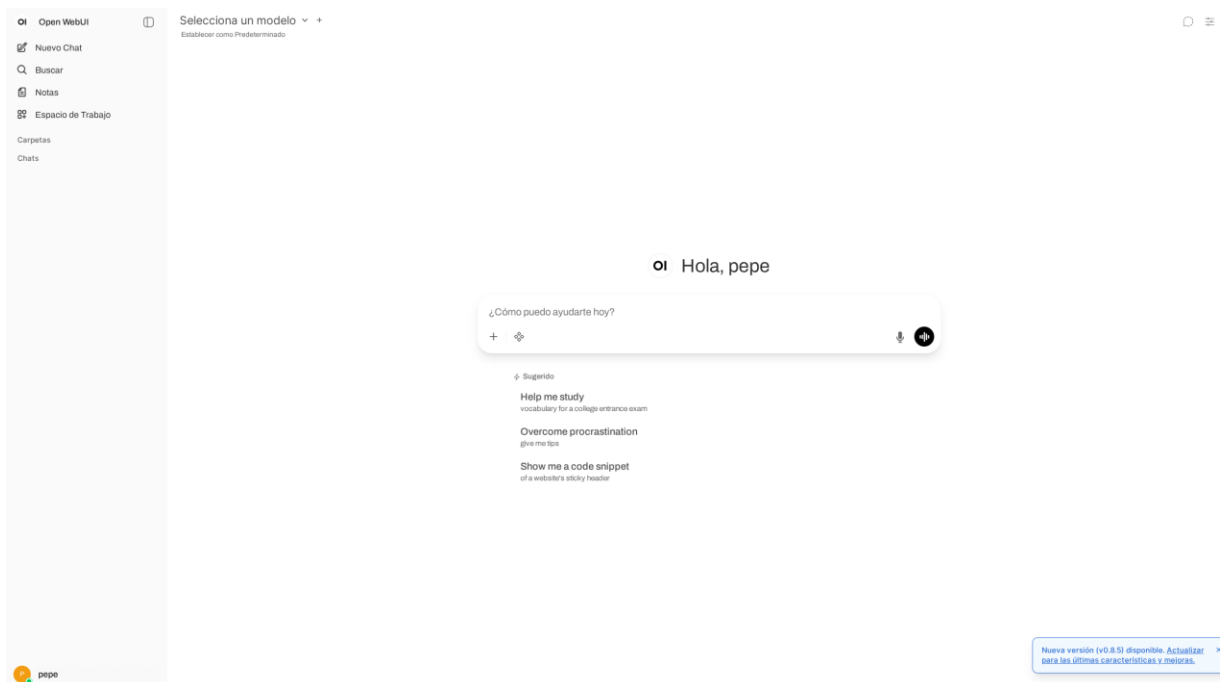


Figura 1 Pantalla principal de OpenUI.

Towards an AI co-scientist

El artículo presenta el AI co-scientist, un sistema multi-agente basado en Gemini 2.0 que asiste en el descubrimiento científico generando hipótesis originales y verificables mediante un ciclo de generación, debate y evolución. Opera bajo un paradigma “scientist-in-the-loop”, donde científicos revisan y validan las propuestas fundamentadas en literatura. Validado en biomédica, identificó candidatos para leucemia, nuevas dianas terapéuticas y mecanismos evolutivos, superando a otros LLM en calidad de hipótesis y evidencia experimental.

Construcción de un grafo de conocimiento con LlamaCloud y Neo4J

Contexto y motivación

La técnica de Retrieval-Augmented Generation (RAG) mejora significativamente las capacidades de los modelos de lenguaje al complementarlos con fuentes externas de conocimiento. Sin embargo, la búsqueda por similitud semántica convencional puede fallar al capturar relaciones complejas y contextos distribuidos en múltiples documentos. Transformar documentos no estructurados en grafos de conocimiento permite realizar consultas relacionales más ricas, razonamiento contextual y travesías complejas que superan las limitaciones de la búsqueda vectorial simple.

Las herramientas principales empleadas son LlamaCloud, que integra herramientas como LlamaClassify, que clasifica documentos (por ejemplo, contratos legales), y LlamaExtract, que extrae atributos relevantes según el tipo de documento identificado, y Neo4j, utilizado como plataforma para almacenar y representar el grafo de conocimiento resultante.



Figura 2 Logo de llama-index.



Figura 3 Logo de neo4j.

Figura 4 Principales tecnologías empleadas.

Flujo de trabajo (pipeline)

El ejemplo guía a través de un flujo completo aplicado al procesamiento de documentos legales, con las siguientes etapas:

- Instalación y configuración: se emplean librerías como llama-index-workflows, llama-cloud-services, jsonschema, openai, neo4j y llama-index-llms-openai.
- Clasificación de documentos con LlamaClassify para determinar la categoría de cada contrato.
- Extracción de información usando LlamaExtract, según los atributos relevantes a cada tipo de contrato.
- Almacenamiento en Neo4j: transformación de la información estructurada en nodos y relaciones dentro de un grafo de conocimiento.

Aplicación y beneficios

- **Modelado relacional:** los grafos permiten visualizar y analizar cómo los distintos elementos del documento se relacionan, lo que posibilita consultas más sofisticadas mediante técnicas como GraphRAG.
- **Dominio legal como caso de uso:** en los documentos legales, donde las relaciones entre cláusulas, referencias y entidades son esenciales, esta aproximación mejora notablemente la precisión y contextualización.
- **Ventajas técnicas:** Permite consultas de datos directos como relaciones indirectas en un contexto más profundo. Además, facilita el razonamiento estructurado más allá de similitud superficial.

La construcción de un grafo de conocimiento a partir de documentos no estructurados mediante el uso combinado de LlamaCloud y Neo4j representa un avance significativo en la integración de inteligencia artificial y bases de datos orientadas a grafos. Al superar las limitaciones de la búsqueda semántica tradicional, esta solución permite consultas más complejas, razonamiento contextual profundo y una representación estructurada del conocimiento.

Particularmente en el dominio legal, donde las relaciones entre cláusulas, entidades y referencias son críticas, este enfoque aporta una mayor precisión en la interpretación y recuperación de información. El uso de herramientas como LlamaClassify y LlamaExtract automatiza el procesamiento documental, mientras que Neo4j facilita la persistencia y navegación eficiente del conocimiento extraído.

En conjunto, este pipeline no solo mejora la calidad de los sistemas RAG mediante técnicas como GraphRAG, sino que sienta las bases para desarrollos más avanzados en comprensión de documentos, generación de respuestas enriquecidas y soporte a la toma de decisiones informadas en dominios complejos.

LangChain y LlamaIndex

Para seleccionar el *framework* que servirá como interfaz entre el modelo de lenguaje grande (LLM) y la base de datos, se consideraron las dos principales alternativas disponibles en el mercado: LangChain y LlamaIndex.

LangChain constituye un *framework* de propósito general, no especializado en ningún ámbito particular. Se caracteriza por ofrecer una amplia gama de herramientas aplicables a diversos componentes de un sistema de inteligencia artificial (desde el LLM hasta agentes o gestión de memoria, entre otros). Dispone de conectores para el manejo adecuado de los datos, aunque su principal ventaja frente a LlamaIndex radica en su flexibilidad, un aspecto que podría resultar valioso para gestionar la complejidad integral del proyecto, dada la diversidad de ramas y casos de uso que se están desarrollando.

Por su parte, LlamaIndex es también un *framework* orientado al trabajo con LLMs, pero con una diferencia fundamental: se trata de una solución especializada. Está específicamente diseñado para el consumo y consulta de datos, y, de hecho, opera sobre LangChain en su capa subyacente. Ofrece una amplia variedad de funciones preconstruidas que facilitan notablemente la implementación. Además, está concebido para proporcionar información a partir de datos previamente suministrados, lo que lo convierte en una opción idónea para nuestro caso de uso.

Una analogía apropiada en términos de lenguajes de programación sería comparar Java, un lenguaje de alto nivel, con C, uno más versátil y que permite un control más granular.

En virtud de su mayor simplicidad de implementación, su enfoque específico en la presentación de información a partir de datos almacenados y la reducción en la cantidad de instrucciones requeridas para su correcto funcionamiento, se ha seleccionado LlamaIndex como el *framework* para este proyecto.

N8N

Se estudió el uso de N8N en el proyecto, una plataforma de automatización de flujos de trabajo basada en bajo código. El término “bajo código” hace referencia a un enfoque de desarrollo que minimiza la necesidad de escribir código manualmente, utilizando herramientas visuales, plantillas y componentes predefinidos para construir aplicaciones o procesos de manera más rápida y accesible. Cada nodo representa una acción concreta, como enviar un correo, escribir en una base de datos o ejecutar un script personalizado, lo que permite encadenar tareas de forma automatizada.

Durante el análisis se realizaron pruebas exhaustivas de sus capacidades aplicables al proyecto. Además, se exploró la ejecución local de proyectos mediante Docker sin necesidad de licencias.

No obstante, durante la evaluación se identificaron limitaciones significativas. El enfoque de bajo código, aunque simplifica la adopción, reduce la versatilidad, ya que ciertas operaciones avanzadas requieren soluciones menos directas o no son posibles dentro del entorno visual. Asimismo, depender de la interfaz gráfica limita la potencia y eficiencia frente a alternativas basadas en CLI, donde la programación directa ofrece mayor control, flexibilidad y capacidad de integración con entornos complejos.

Respecto a su implementación concreta en el proyecto, N8N era técnicamente viable. Sin embargo, el modelo de licencias supuso un obstáculo crítico: aunque es de código abierto y permite uso privado, exige la adquisición de una licencia para cualquier fin comercial, incluyendo proyectos sin ánimo de lucro destinados a ofrecer servicios públicos. En consecuencia, se decidió relegar su uso al desarrollo de herramientas

internas que no requieran licencia comercial, aprovechando su facilidad de uso sin comprometer la viabilidad del proyecto.

MCP

El Model Context Protocol (MCP) es un estándar abierto de protocolos que permite a las aplicaciones de inteligencia artificial interactuar de manera dinámica con herramientas y fuentes de información externas al sistema. Su principal objetivo es facilitar el acceso seguro y organizado a información en tiempo real y contexto específico, mejorando así las capacidades de los modelos de lenguaje.

Entre las ventajas principales asociadas al uso de MCP se encuentran la capacidad de razonamiento basado en datos actualizados, lo que reduce significativamente la posibilidad de alucinaciones y errores asociados con información desactualizada, así como su escalabilidad.

No obstante, también presenta aspectos negativos. Aunque MCP fue diseñado considerando aspectos de seguridad, su implementación implica conectar el modelo de lenguaje (LLM) a servidores externos, lo que introduce puntos adicionales de acceso y posibles vectores de vulnerabilidad si no se configura adecuadamente. Adicionalmente, cuenta con un sistema de gestión de permisos limitado, cuyo control granular aún se encuentra en desarrollo.

Asimismo, al tratarse de una tecnología emergente, cuenta con un ecosistema aún incipiente, lo que implica una trayectoria de evolución incierta y dificulta prever su adopción y mejora continua.

Considerando los riesgos potenciales en materia de seguridad y privacidad, se ha determinado no adoptar esta tecnología en el estado actual de su desarrollo.

A2A y MoE

En el estudio de servicios para el proyecto se analizó el paradigma Agent-to-Agent (A2A), que se basa en la interoperabilidad entre agentes de software autónomos y heterogéneos mediante protocolos de comunicación estandarizados. Esta arquitectura permite que distintos sistemas —como APIs, servicios de computación o modelos de lenguaje— se descubran, negocien tareas y coordinen acciones de manera dinámica. Desde la perspectiva del proyecto, A2A ofrece un alto potencial de modularidad y escalabilidad, ya que permite integrar agentes independientes sin modificar la arquitectura global, e incluso combinar LLM de diferentes proveedores. Además, la comunicación basada en APIs o protocolos de mensajería facilita la extensión de funcionalidades y la integración de nuevos agentes en el futuro.

Sin embargo, la implementación de un sistema A2A presenta limitaciones relevantes. Su complejidad de coordinación requiere un diseño cuidadoso y protocolos robustos, y la comunicación externa entre agentes es más lenta que la comunicación interna de un único sistema. En el contexto del proyecto, que no se centraliza sobre plataformas como n8n, establecer un sistema multi-agente resultaría complicado y costoso en términos de desarrollo.

Por estas razones, se considera que A2A es un enfoque interesante a largo plazo, pero no viable ni prioritario en la fase actual del proyecto.

En paralelo, se evaluó la arquitectura Mixture of Experts (MoE) utilizada en modelos de lenguaje avanzados como DeepSeek. MoE distribuye la carga computacional entre múltiples subredes especializadas o “expertos”, activando únicamente aquellos necesarios para cada tarea mediante un gating network. Esta técnica permite optimizar la eficiencia interna del modelo y su escalabilidad, sin incrementar excesivamente los costos de inferencia. En el proyecto, la arquitectura MoE se adapta bien a la tecnología del LLM empleado, garantizando un rendimiento estable y sin problemas de procesamiento. No obstante, su integración aumenta la complejidad del diseño y la gestión del modelo, no ofrecen la flexibilidad de agentes independientes como en A2A.

En conclusión, A2A y MoE se mantienen como alternativas a considerar en el futuro: A2A si se requiere modularidad y colaboración entre múltiples sistemas, y MoE si se necesita optimizar la eficiencia interna del modelo. Esta aproximación permite mantener el proyecto viable y controlable, sin comprometer su rendimiento ni su escalabilidad a corto plazo.

Formación

El equipo del proyecto ha completado varias formaciones de las tecnologías a usar en el proyecto. Entre las plataformas empleadas destacan Neo4j Academy y DeepLearning. A parte de realizar varias investigaciones sobre las tecnologías a usar en el proyecto como podrían ser N8N, MCP, A2A y sobre varias publicaciones académicas. Además, parte del equipo acudió a la formación del Cybersecurity Summer Bootcamp.

Metodología y gestión del proyecto

El desarrollo de MARC.0 ha seguido una metodología ágil adaptada a un entorno de investigación académica. Se ha empleado un repositorio centralizado de código (InfraestructuraDigiliencia) con control de versiones en GitHub.

Cabe destacar que el cronograma del proyecto se vio impactado significativamente por factores externos a la labor de desarrollo de software. Específicamente, la disponibilidad de la infraestructura en SCAYLE sufrió una demora de más de seis meses. Este retraso obligó al equipo a simular entornos de alta carga en máquinas locales durante gran parte del desarrollo, retrasando el desarrollo del proyecto. Una vez provisionados los recursos en el SCAYLE se realizó una migración del sistema a la nueva infraestructura.

Asimismo, el equipo humano ha sufrido reestructuraciones durante el ciclo de vida del proyecto. La salida del desarrollador Esteban Peña Salazar requirió una redistribución de tareas, especialmente en lo referente al mantenimiento de los scripts de ingesta y la configuración inicial de la base de datos, tareas que fueron asumidas por el resto de los miembros del equipo.

Diseño de la arquitectura

El sistema se ha diseñado bajo un modelo de arquitectura en tres capas (presentación, lógica de negocio y persistencia), articulado mediante un esquema cliente-servidor. Esta disposición representa el estándar de facto en servicios web y permite una separación clara de responsabilidades entre el *frontend* y el *backend*. Para garantizar la integridad operativa y la portabilidad, los módulos se encuentran aislados tanto lógicamente como físicamente mediante el uso de contenedores Docker.

El diseño arquitectónico se fundamenta en los pilares estratégicos de soberanía de datos e independencia tecnológica. En virtud de estos principios, se ha priorizado una infraestructura local donde la totalidad de los servicios se ejecutan en servidores propios. Esta decisión minimiza la dependencia de proveedores externos y asegura un control exhaustivo sobre el ciclo de vida del *software* y el *hardware*.

El sistema se divide en los siguientes módulos:

- **Extracción de Datos (Scraping):** Este módulo se encarga de la captura y el procesamiento de información externa para mantener actualizado el dataverso. Su función es garantizar que la base de conocimientos cuente con datos actualizados.
- **Persistencia Híbrida:** El almacenamiento se gestiona mediante dos sistemas. Se utiliza una base de datos SQL para la administración de usuarios y datos generales de la aplicación, y una base de datos orientada a grafos para organizar y consultar el dataverso de forma eficiente.
- **Modelo de Lenguaje (LLM):** Es el componente central para el procesamiento de texto. Se ejecuta localmente mediante Ollama para asegurar la autonomía del sistema y la privacidad de la información procesada.

- **Orquestador:** Este módulo coordina un conjunto de agentes de IA. Su función es gestionar el acceso al dataverso y a otras herramientas para controlar la generación de respuestas del modelo, mejorando su precisión. Constituye el elemento diferenciador del proyecto.
- **Lógica de Negocio y API:** Implementa las reglas del servicio y gestiona el acceso mediante una interfaz REST. Actúa como el puente de comunicación entre los procesos internos y el exterior.
- **Interfaz de Usuario (Frontend):** Consiste en una aplicación web que permite al usuario interactuar con el sistema. Se comunica con el servidor a través de la API para enviar peticiones y recibir resultados.

En la figura 5 se muestra un esquema de los módulos principales del proyecto.

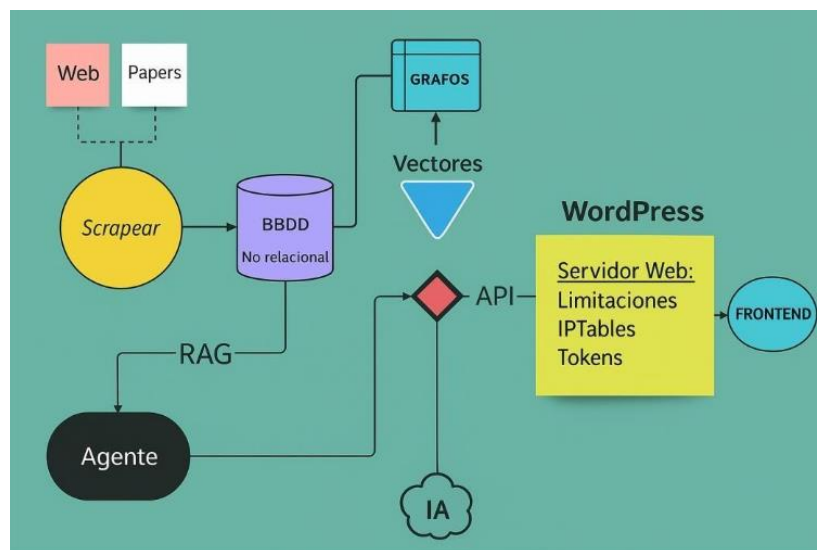


Figura 5 Esquema general de la arquitectura del proyecto.

Subsistema de Persistencia de Datos

La naturaleza heterogénea de la información tratada en MARC.0 ha requerido la implementación de dos bases de datos diferentes: la base de datos SQL PostgreSQL para la gestión de los datos generados por la aplicación (conversaciones de la IA y gestión de usuarios) y la base de datos orientada a grafos Neo4J para la información extraída del proceso de *scrapping*.

Modelado de Grafos en Neo4j

El núcleo de inteligencia reside en una base de datos orientada a grafos Neo4j. A diferencia de los modelos relacionales, que requieren costosas operaciones de JOIN para vincular tablas, el modelo de grafos almacena las relaciones como objetos de primera clase. Esta información forma nuestro dataverso.

El dataverso es la colección de datos, información, que el modelo de inteligencia artificial empleará como fuente de conocimiento para generar sus respuestas. Es el corazón de proyecto y fundamental en la calidad del servicio.

La estructura de datos no es rígida, pero se adhiere a una ontología definida para garantizar la consistencia semántica:

- **Nodos (Label):**

- Article: Entidad central que contiene el cuerpo del texto, metadatos temporales y vector de embedding.
- ThreatActor: Identifica grupos o individuos responsables de ataques.
- Vulnerability: Mapea identificadores CVE (*Common Vulnerabilities and Exposures*).
- Sector: Categorización industrial (Financiero, Salud, Energía).

- **Relaciones (Type):**

- TARGETS: Dirección del ataque (ThreatActor -> Sector).
- EXPLOITS: Uso de vulnerabilidad (ThreatActor -> Vulnerability).
- MENTIONS: Relación semántica extraída (Article -> ThreatActor).

Taxonomía de Clasificación

El sistema define las siguientes temáticas de ciberseguridad: Adware, Botnet, Cyberbullying, Cybersquatting, Deepfake, Defacement, Grooming, Jailbreaking, Malvertising, Malware, Man-in-the-middle, Pentesting, Phishing, Pretexting, Ransomware, Rooting, Sexting, Sextorsion, Smishing, Social engineering, Spear phishing, Spyware, Typosquatting, Vishing, Vulnerability, Warshipping.

Los campos relacionados están organizados como un sistema jerárquico de categorías y subcampos. Algunas de las categorías principales son:

- Internet Governance (7 subcampos: Online Surveillance, Data Protection, Identity Management, etc.)
- Artificial Intelligence (8 subcampos: Generative AI, Responsible AI, AI and Jobs, etc.)
- International Security (7 subcampos: Hybrid Threats, The Technological Arms Race, etc.)
- Illicit Economy (7 subcampos: Cybercrime, Illicit Trade and Financial Flow, etc.)
- Digital Identity (5 subcampos: Identity Fraud and Cyber Resilience, etc.)
- Blockchain (8 subcampos: Blockchain Security and Interoperability, Smart Contracts and Automation, etc.)
- Y más de 30 categorías adicionales.

Extracción de datos

Se han desarrollado un total de ocho programas automatizados de rastreo web (*web crawlers*), cada uno orientado a la recopilación de información desde distintas fuentes especializadas en ciberseguridad. Estos programas están diseñados bajo una arquitectura modular, lo que permite organizar sus funcionalidades en componentes independientes, facilitando tanto su mantenimiento como su escalabilidad.

Los sistemas de rastreo implementados han sido configurados específicamente para operar sobre los siguientes sitios web de referencia en el ámbito de la ciberseguridad:

- Instituto Nacional de Ciberseguridad
- Centro Criptológico Nacional
- National Cyber Security Centre
- World Economic Forum
- Canadian Centre for Cyber Security
- National Institute of Standards and Technology
- Cybersecurity and Infrastructure Security Agency

Descripción de los scrapers

Scraper de Instituto Nacional de Ciberseguridad

Este scraper se centra en la recopilación de avisos de seguridad, noticias y publicaciones divulgativas. Se ha adaptado a la estructura del portal, identificando categorías como actualidad, avisos y blogs. La extracción incluye metadatos relevantes como fecha de publicación, título y contenido completo, priorizando información reciente relacionada con vulnerabilidades y buenas prácticas.

Scraper de Centro Criptológico Nacional

El programa desarrollado para este sitio está orientado a la obtención de informes técnicos, alertas y boletines de seguridad. Debido a la naturaleza estructurada de sus publicaciones, el scraper implementa mecanismos específicos para acceder a documentos oficiales y extraer información tanto de páginas HTML como de documentos enlazados.

Scraper de National Cyber Security Centre

Este scraper se enfoca en guías de seguridad, noticias y alertas públicas. Se ha diseñado para navegar por las distintas secciones temáticas del sitio, extrayendo contenido técnico y recomendaciones. Además, contempla la jerarquía del sitio para acceder a subcategorías relevantes.

Scraper de World Economic Forum

El rastreador del World Economic Forum se centra en artículos, informes y publicaciones relacionadas con ciberseguridad dentro de un contexto global. Dado el carácter multidisciplinar del sitio, el scraper filtra contenido relevante mediante palabras clave y categorías específicas.

Problemática y Solución en Web Forum (World Economic Forum)

Uno de los mayores desafíos técnicos encontrados durante la implementación fue la extracción de datos del **World Economic Forum (WeForum)**.

Durante las pruebas iniciales, los *scripts* basados en *requests* devolvían contenido vacío o incompleto. Se detectó que este portal utiliza una carga diferida de contenido (lazy loading) agresiva y renderizado en el lado del cliente (Client-Side Rendering) mediante frameworks de JavaScript modernos. Además, implementan medidas anti-bot que bloquean peticiones sin cabeceras de navegador legítimas (User-Agent).

Como solución se reescribió el módulo `weforum_scraper.py` para utilizar una instancia de navegador completa gestionada por Selenium. Se configuró el webdriver sin en el modo headless, es decir, con interfaz gráfica para su ejecución en los servidores de SCAYLE (que carecen de entorno de escritorio). Después, se implementaron esperas explícitas (`WebDriverWait`) vinculadas a la presencia de elementos específicos del DOM (títulos de artículos, cuerpos de texto) antes de intentar la extracción. También se añadieron retardos aleatorios entre peticiones para simular comportamiento humano y evitar el bloqueo de la IP desde la que se realizada el *scraping*.

Scraper de Canadian Centre for Cyber Security

Este programa está diseñado para extraer alertas, publicaciones técnicas y guías de seguridad del organismo canadiense. Se ha optimizado para tratar estructuras de contenido estandarizadas, permitiendo una extracción eficiente y consistente de datos.

Scraper de National Institute of Standards and Technology

El scraper de NIST se orienta principalmente a la recopilación de publicaciones académicas, estándares y documentos técnicos. Debido a la complejidad y volumen de

información, se implementan filtros para seleccionar contenido relevante en el ámbito de la ciberseguridad.

Scraper de Cybersecurity and Infrastructure Security Agency

Este rastreador se centra en la obtención de alertas, recomendaciones y noticias relacionadas con la seguridad de infraestructuras críticas. Se ha diseñado para identificar rápidamente contenido prioritario, como avisos urgentes o vulnerabilidades activas.

Arquitectura funcional

La mayoría de los rastreadores desarrollados comparten una estructura de implementación homogénea, basada en dos funcionalidades principales:

1. Módulo de rastreo y extracción de contenido

Este módulo se encarga de localizar y recopilar información relevante, como artículos, noticias, publicaciones técnicas, eventos y conferencias. Para ello, identifica los enlaces a los contenidos de interés y realiza la extracción de los datos asociados (título, fecha, autor, resumen, cuerpo del texto, entre otros).

2. Módulo de exploración de secciones

Este componente se centra en recorrer las distintas secciones internas de cada sitio web, tales como categorías, archivos históricos o apartados temáticos, garantizando una cobertura completa del contenido disponible.

Ejecución y configuración

Los programas han sido diseñados para permitir su ejecución concurrente, lo que posibilita que múltiples procesos de rastreo se ejecuten de forma simultánea, optimizando el rendimiento global del sistema.

La ejecución de cada programa está controlada mediante un parámetro configurable que define el número de días hacia atrás que se desea extraer información, lo que permite adaptar el volumen de datos procesados a las necesidades específicas.

Escalabilidad y adaptabilidad

La arquitectura propuesta facilita una ejecución dinámica y flexible del sistema. Gracias a su diseño modular y parametrizable, es posible ajustar la frecuencia de rastreo, incorporar nuevas fuentes y escalar el sistema según los recursos disponibles.

En conjunto, esta solución permite construir un sistema robusto y eficiente para la recopilación automatizada de información en ciberseguridad, garantizando tanto la calidad como la actualidad de los datos obtenidos.

Normalización y Preprocesamiento

Antes de la inserción en la base de datos, todo el texto extraído pasa por una tubería (*pipeline*) de limpieza:

- Eliminación de etiquetas HTML residuales.
- Normalización de fechas al formato ISO 8601 (crítico para las consultas temporales en Neo4j).
- Deduplicación: Uso del título y URL como claves únicas para evitar que la ejecución recurrente de los *scrapers* genere información redundante.

Arquitectura RAG (Retrieval-Augmented Generation)

El sistema sigue una arquitectura multi-agente con memoria compartida. Un agente orquestador (RouterAgent) recibe las consultas del usuario, decide qué agente especializado debe procesarla y delega el trabajo. Todos los agentes comparten un contexto de conversación común a través de una memoria centralizada.

El proceso de generación de la respuesta a una consulta del usuario es la siguiente:

- **Recepción de la Consulta:** El usuario formula una pregunta en lenguaje natural.
- **Recuperación de Contexto (Retrieval):** El sistema convierte la pregunta en un vector y consulta a Neo4j los \$k\$ fragmentos de texto más relevantes a través del sistema de agentes.
- **Aumento del Prompt (Augmentation):** Se construye dinámicamente un *prompt* de sistema que inyecta los fragmentos recuperados. La instrucción enviada al modelo es explícita: *"Utiliza exclusivamente el siguiente contexto para responder. Si la respuesta no está en el contexto, indica que no tienes información."*
- **Generación:** El modelo de lenguaje (LLM) sintetiza la respuesta final citando las fuentes.

La comunicación con el modelo de lenguaje se realiza a través de Ollama, empleado como servidor de inferencia local. El framework de orquestación utilizado es LlamaIndex. Se han empleado los modelos gpt-oss:120b, gemma3:27b y qwen3:32b, siendo este último el mejor en cuanto a calidad y rendimiento, ya puede ejecutarse plenamente en la GPU del servidor.

Los componentes principales del sistema de agentes son:

- RouterAgent: Orquestador que clasifica y enruta las consultas.
- ConversationalAgent: Agente para preguntas generales y educación en ciberseguridad.
- NewsAgent: Agente RAG especializado en noticias de ciberseguridad con acceso a herramientas y búsqueda semántica.
- SharedConversationMemory: Memoria de conversación compartida por todos los agentes.
- AgentManager: Gestor de sesiones que crea, cachea y limpia instancias de agentes por conversación.
- AgentFactory: Factoría centralizada para la creación de agentes.

A continuación, se detallan más detenidamente.

Jerarquía de Agentes

Clase Base: BaseAgent

- Módulo: agents/base_agent.py
- Tipo: Clase abstracta.
- Responsabilidad: Define la interfaz y funcionalidad común que todo agente debe implementar.
- Atributos principales:
 - name (str): Identificador único del agente.
 - description (str): Descripción de las capacidades del agente.
 - model_name (str): Nombre del modelo Ollama a utilizar (por defecto: "llama3.1:8b").
 - temperature (float): Temperatura del LLM. Rango 0.0 (determinístico) a 1.0 (creativo).
 - request_timeout (float): Tiempo máximo de espera para la respuesta del LLM (por defecto: 600 segundos = 10 minutos).
 - _shared_memory (SharedConversationMemory): Referencia a la memoria compartida.
 - _llm (LLM): Instancia del modelo de lenguaje Ollama, inicializada en el constructor.
- Métodos abstractos (obligatorios para subclases):
 - process_query(query: str, **kwargs) -> str: Procesa la consulta del usuario y devuelve una respuesta.
 - get_system_prompt() -> str: Devuelve el prompt de sistema del agente.
- Métodos concretos:

- `_initialize_llm()` -> LLM: Crea una instancia de Ollama con los parámetros configurados (modelo, temperatura, timeout).
- `build_prompt_with_context(query, system_prompt)` -> str: Construye un prompt completo que incorpora el historial de conversación desde la memoria compartida, añadiendo instrucciones para que el LLM utilice el contexto previo.
- `get_conversation_context()` -> str: Recupera el contexto de conversación formateado desde la memoria compartida.
- `record_query(success: bool)`: Registra métricas de éxito/fallo.
- `get_metrics()` -> Dict: Devuelve estadísticas de rendimiento (total de consultas, exitosas, fallidas, tasa de éxito).

La conexión al modelo se realiza mediante la clase Ollama de LlamaIndex:

```
llm = Ollama(  
    model=self.model_name,      # e.g. "llama3.1:8b"  
    temperature=self.temperature, # e.g. 0.7  
    request_timeout=self.request_timeout, # e.g. 600.0  
)
```

Clase Intermedia: `ConversationalAgentBase`

- Módulo: `agents/base_agent.py`
- Hereda de: `BaseAgent`
- Responsabilidad: Base para agentes conversacionales que no requieren herramientas externas.
- Funcionalidad adicional:
 - Mantiene un historial de conversación local (`_conversation_history`) como lista de diccionarios `{"role": ..., "content": ...}`.
 - El método `add_to_history(role, content)` registra los mensajes tanto en el historial local como en la memoria compartida. Los mensajes del asistente se registran con el nombre del agente.
 - Métodos `clear_history()` y `get_history()` para gestionar el historial local.

`ConversationalAgent`

- Módulo: `agents/conversational_agent.py`
- Hereda de: `ConversationalAgentBase`
- Responsabilidad: Atender preguntas generales, explicaciones y educación en ciberseguridad sin acceso a herramientas externas.
- Configuración:
 - Temperatura por defecto: 0.7 (más creativo, adecuado para conversación).

- Motor de chat: SimpleChatEngine de LlamaIndex, inicializado con el LLM y el prompt de sistema.
- Flujo de procesamiento (process_query):
- Obtiene el contexto de conversación desde la memoria compartida (get_conversation_context()).
- Si existe contexto previo, construye una consulta mejorada que incluye el historial de conversación con la instrucción de considerar la discusión previa.
- Envía la consulta mejorada al SimpleChatEngine (self._chat_engine.chat(enhanced_query)).
- Registra el mensaje del usuario y la respuesta del asistente en el historial local y en la memoria compartida.
- Devuelve la respuesta como texto.

Prompt de sistema:

El CONVERSATIONAL_AGENT_SYSTEM_PROMPT define al agente como un asistente de educación en ciberseguridad para INCIBE con las siguientes directrices:

- Explicar conceptos de ciberseguridad en términos simples y accesibles.
- Proporcionar consejos prácticos de seguridad.
- No buscar ni proporcionar noticias específicas (eso corresponde al NewsAgent).
- No inventar estadísticas ni datos actuales.
- Tono amigable, paciente, educativo y profesional.
- Temáticas: seguridad de contraseñas, navegación segura, tipos de malware, ingeniería social, protección de privacidad, seguridad de dispositivos y redes, autenticación de doble factor, etc.

NewsAgent (Agente RAG)

- Módulo: agents/rag/news_agent.py
- Hereda de: BaseAgent
- Responsabilidad: Recuperar y presentar noticias de ciberseguridad desde la base de datos de grafos Neo4j, utilizando herramientas de búsqueda y un sistema RAG con embeddings vectoriales.
- Configuración:
 - Temperatura por defecto: 0.0 (determinístico, para respuestas factuales).
 - Motor de agente: ReActAgent de LlamaIndex con un máximo de 15 iteraciones.
 - Timeout de las peticiones al LLM: 7 minutos (420 segundos).
- Inicialización:

- Configura un modelo de embeddings personalizado (CustomAPIEmbedding) que se conecta a un servicio externo de embeddings o a Ollama.
- Establece Settings.embed_model y Settings.llm de LlamaIndex globalmente para evitar que LlamaIndex intente utilizar modelos de OpenAI como fallback.
- Crea un ReActAgent con las herramientas de recuperación de noticias.

Herramientas disponibles (Tools):

El NewsAgent dispone de dos herramientas:

- get_news (FunctionTool):
- Tipo: Herramienta de función directa.
- Descripción: Recupera noticias de la base de datos Neo4j mediante consultas Cypher filtradas.
- Parámetros:
- limit (int, max 50): Número máximo de noticias.
- topic (str, opcional): Filtrar por temática (Ransomware, Phishing, Malware, etc.).
- related_field (str, opcional): Filtrar por campo relacionado.
- start_date (str, opcional): Fecha de inicio (formato YYYY-MM-DD).
- end_date (str, opcional): Fecha de fin (formato YYYY-MM-DD).
- organization (str, opcional): Filtrar por agencia de noticias.
- Funcionamiento interno: Valida los parámetros contra los enums Topics y RelatedFields, construye una consulta Cypher dinámica con los filtros activos y la ejecuta contra Neo4j. Devuelve los resultados como texto formateado incluyendo título, fecha, URL, contenido, agencia, temas y campos.
- semantic_news_search (QueryEngineTool):
- Tipo: Herramienta de motor de consulta sobre un vector store.
- Descripción: Realiza búsqueda semántica sobre los chunks de noticias almacenados en Neo4j con embeddings vectoriales.
- Funcionamiento interno:
- Se conecta a Neo4jVectorStore configurado con un índice vectorial (chunk_embedding_idx) y búsqueda híbrida habilitada.
- Utiliza una consulta Cypher personalizada (retrieval_query) que:
- Localiza los nodos Chunk más relevantes semánticamente.
- Recorre la estructura del grafo para obtener el contexto completo: la noticia padre (News), la agencia (NewsAgency), los autores (Author), los temas (Topic) y los campos relacionados (Field).
- Devuelve el texto del chunk junto con metadatos enriquecidos: título de la noticia, fecha, URL, preview del contenido, agencia, autores, temas y campos.
- Crea un VectorStoreIndex a partir del store y lo expone como query_engine.

Flujo de procesamiento (process_query):

El flujo de procesamiento de una consulta es la siguiente:

- Obtiene el contexto de conversación desde la memoria compartida.
- Si existe contexto previo, enriquece la consulta con el historial.
- Ejecuta la consulta de forma asíncrona mediante el ReActAgent.
- El ReActAgent decide de forma autónoma qué herramientas(s) utilizar y cuántas veces invocarlas (hasta 15 iteraciones).
- Extrae el texto de la respuesta final, filtrando los bloques de razonamiento ("thinking"/"reasoning") del LLM.
- Registra la respuesta en la memoria compartida.
- Devuelve la respuesta como texto.

Prompt de sistema del NewsAgent:

El NEWS_AGENT_SYSTEM_PROMPT define al agente como un agente especializado de inteligencia de noticias para INCIBE con las siguientes directrices:

- Siempre debe utilizar las herramientas para buscar y recuperar datos de noticias.
- Nunca debe inventar o alucinar información.
- Basar todas las respuestas estrictamente en los datos devueltos por las herramientas.
- Si las herramientas no devuelven resultados, indicarlo claramente.
- Presentar las noticias en formato estructurado: título, fecha, organización, resumen y relevancia.
- Proporcionar contexto sobre por qué la noticia importa a los ciudadanos españoles.
- Tono profesional pero accesible.
- Temáticas: ataques ransomware, brechas de datos, vulnerabilidades de software, campañas de phishing, ataques a infraestructuras, amenazas emergentes, avisos de seguridad.
- Incluye la fecha actual para contexto temporal.

RouterAgent (Orquestador)

- Módulo: agents/router_agent.py
- Hereda de: BaseAgent
- Responsabilidad: Analizar las consultas del usuario, clasificarlas y enrutarlas al agente especializado apropiado.
- Configuración:
 - Temperatura por defecto: 0.1 (muy baja, para decisiones de routing consistentes y determinísticas).
 - Tipos de agente disponibles (enum AgentType):
 - NEWS_AGENT

- **CONVERSATIONAL_AGENT**

- Inicialización:
- Crea una instancia de SharedConversationMemory propia (o usa la proporcionada).
- Inicializa ambos agentes especializados (NewsAgent y ConversationalAgent) inyectándoles la misma referencia a la memoria compartida.
- Inicializa estadísticas de routing por tipo de agente.
- Flujo de routing (`_route_query`):
- Obtiene el contexto de conversación de la memoria compartida.
- Construye un prompt de routing que incluye:
- El prompt de sistema del router (instrucciones de clasificación).
- El historial de conversación (si existe), con la indicación de considerar el contexto previo para enrutar al mismo agente si el usuario da seguimiento a un tema anterior.
- La consulta del usuario.
- Envía el prompt al LLM (`self._llm.complete(routing_prompt)`).
- Parsea la respuesta del LLM:
- Si contiene "NEWS_AGENT" o "NEWS", entonces `AgentType.NEWS_AGENT`.
- Si contiene "CONVERSATIONAL_AGENT" o "CONVERSATIONAL", entonces `AgentType.CONVERSATIONAL_AGENT`.
- En caso ambiguo o error → por defecto `CONVERSATIONAL_AGENT`.
- Si la respuesta del LLM contiene texto de razonamiento, extrae solo la palabra/línea final.
- Flujo de procesamiento (`process_query`):
- Registra el mensaje del usuario en la memoria compartida.
- Llama a `_route_query` para determinar el agente destino.
- Actualiza las estadísticas de routing.
- Invoca `process_query` del agente seleccionado.
- El agente elegido procesa la consulta y registra su respuesta en la memoria compartida.
- Retorna la respuesta al llamante.

Prompt de sistema del RouterAgent:

El ROUTER_SYSTEM_PROMPT define las reglas de clasificación:

- El router debe responder únicamente con el nombre del agente (NEWS_AGENT o CONVERSATIONAL_AGENT).
- No debe proporcionar explicaciones, razonamientos ni texto adicional.
- No debe intentar responder la pregunta directamente.
- Para consultas que encajan en ambos agentes, preferir NEWS_AGENT si la consulta es sensible al tiempo o relacionada con eventos.
- En caso de duda, usar CONVERSATIONAL_AGENT como fallback.

Criterios de routing:

- **NEWS_AGENT:** Consultas sobre noticias de ciberseguridad, incidentes, vulnerabilidades, amenazas y eventos actuales. Ejemplos: "¿Cuáles son los últimos ataques de ransomware?", "Muéstrame noticias sobre brechas de datos".
- **CONVERSATIONAL_AGENT:** Preguntas generales, explicaciones, definiciones, consejos y mejores prácticas. Ejemplos: "¿Qué es el phishing?", "¿Cómo puedo proteger mis contraseñas?".

Proceso general

En resumen, el proceso de generación de las respuestas es el siguiente:

- La consulta del usuario llega al AgentManager, que obtiene o crea un RouterAgent para esa conversación (chat_id), con su propia SharedConversationMemory aislada.
- Si es la primera petición de la conversación y existe historial previo en la base de datos, el AgentManager carga ese historial en la memoria compartida antes de crear el agente.
- Se invoca RouterAgent.send_msg(message):
 - El mensaje del usuario se registra en la memoria compartida.
 - Se construye un prompt de routing con el historial de conversación y la consulta.
 - El LLM (Ollama, temperatura 0.1) analiza la consulta y devuelve el nombre del agente destino.
- Si el agente destino es ConversationalAgent:
 - Se construye una consulta enriquecida con el historial de conversación.
 - Se envía al SimpleChatEngine (LlamaIndex) que utiliza el LLM (Ollama, temperatura 0.7) con el prompt de sistema educativo de ciberseguridad.
 - La respuesta se registra en la memoria compartida.
- Si el agente destino es NewsAgent:
 - Se construye una consulta enriquecida con el historial de conversación.
 - Se envía al ReActAgent (LlamaIndex) que utiliza el LLM (Ollama, temperatura 0.0).
 - El ReActAgent decide autónomamente qué herramientas invocar:
 - `get_news`: Ejecuta consultas Cypher filtradas contra Neo4j.
 - `semantic_news_search`: Genera embeddings de la consulta y busca en el índice vectorial de Neo4j los chunks más similares, recuperando el contexto completo del grafo.
 - El ReActAgent puede encadenar múltiples llamadas a herramientas (hasta 15 iteraciones) hasta disponer de información suficiente.
 - Con los datos recuperados, el LLM genera una respuesta basada exclusivamente en la información obtenida.

- Se filtran los bloques de razonamiento y se registra la respuesta en la memoria compartida.
- La respuesta se devuelve al usuario original.

Estrategia de Vectorización (Embeddings)

Para permitir la búsqueda semántica sobre los más de 30.000 artículos recolectados, se implementó un *pipeline* de vectorización. A diferencia de la búsqueda por palabras clave (*keyword search*), que falla si no se usan los términos exactos, la búsqueda vectorial permite encontrar conceptos relacionados (ej. buscar "ataque de denegación" y recuperar artículos sobre "DDoS" o "botnets").

- **Modelo Seleccionado:** Se optó por modelos de *embeddings* de código abierto optimizados para el idioma inglés y español, ejecutados localmente para evitar la fuga de datos.
- **Almacenamiento en Neo4j:** Se configuró un índice vectorial (*Vector Index*) dentro de la propia base de datos Neo4j. Esto permite una arquitectura híbrida única: realizar consultas que combinan filtros de grafo (ej. "artículos conectados al nodo 'Sector Financiero'") con similitud semántica (ej. "similares a 'fraude bancario'").

Interfaz API REST

La lógica de negocio del sistema se expone a través de una interfaz de programación de aplicaciones (API) diseñada bajo el estilo arquitectónico REST (Representational State Transfer). Esta capa actúa como el único punto de entrada para el cliente, abstrayendo la complejidad de los procesos internos y garantizando una comunicación estructurada y segura.

El proyecto utiliza FastAPI como *framework*: Un entorno moderno, implementado en Python, que permite crear la API de forma eficiente mediante programación asíncrona (*async/await*).

El diseño sigue una arquitectura modular basada en **routers independientes**, lo que permite organizar los endpoints por dominios funcionales. Cada módulo define sus rutas y dependencias, que posteriormente se registran en la aplicación principal.

La aplicación se inicializa en el módulo `main.py`, donde se configuran los siguientes componentes:

- instancia principal de FastAPI
- middlewares de seguridad
- limitación de tasa de peticiones
- routers de la API

- endpoints globales de servicio

Todos los endpoints se publican bajo el prefijo común "/api".

La estructura general del servicio puede representarse de forma conceptual como se muestra en la Figura 6.

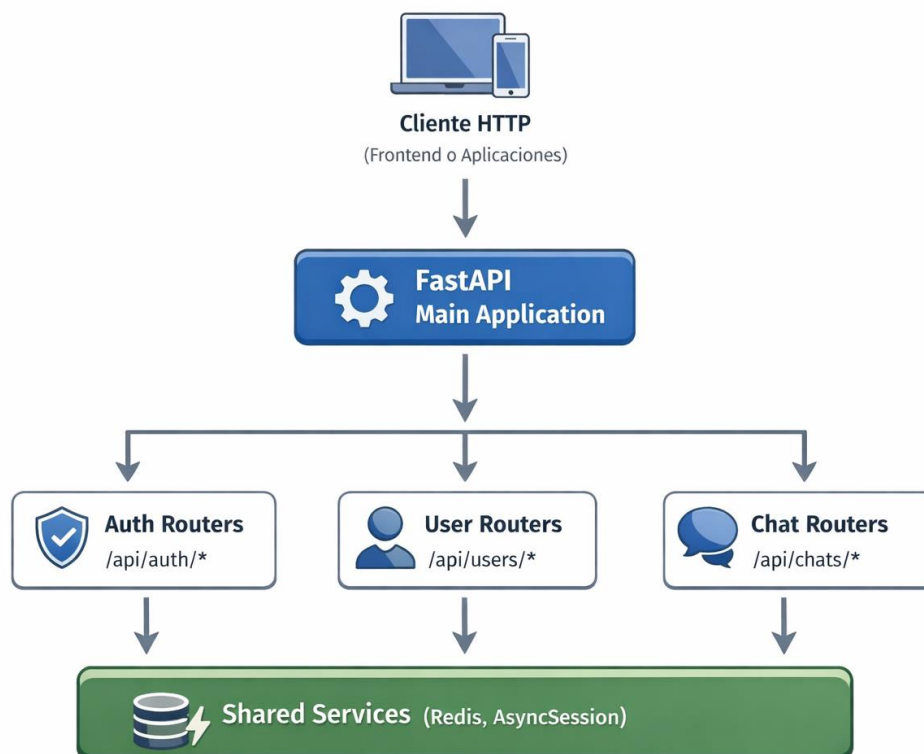


Figura 6: Estructura de la API REST.

La API incorpora dos versiones funcionales:

- **API principal** (/api/...)
- **API v2** (/api/v2/...)

La segunda versión altera los *endpoints* para mejorar la compatibilidad con herramientas externas. En especial, para uno de los clientes *web* desarrollados.

Organización modular

El objetivo principal de la estrategia modular es facilitar el mantenimiento del código, mejorar su legibilidad y permitir la evolución del sistema sin introducir dependencias innecesarias entre componentes.

El *framework* FastAPI proporciona el mecanismo APIRouter, que permite agrupar *endpoints* relacionados dentro de módulos independientes. Cada *router* define sus rutas, dependencias, esquemas de entrada y salida, y lógica de tratamiento de las solicitudes. Posteriormente, estos *routers* se registran en la aplicación principal mediante el método `include_router`, que los integra en el árbol global de rutas del servicio.

De forma conceptual, la estructura de la API se organiza en torno a un conjunto de routers que representan los dominios funcionales del sistema. Entre ellos se encuentran los routers encargados de la autenticación, la gestión de usuarios, la administración de conversaciones, la gestión de plantillas y la selección de modelos. Cada uno de estos módulos implementa exclusivamente las operaciones relacionadas con su dominio, sin asumir responsabilidades pertenecientes a otros componentes.

El directorio routers contiene la implementación principal de los endpoints. Cada archivo define un APIRouter independiente que encapsula las operaciones asociadas a un dominio funcional. Por ejemplo, el router de conversaciones gestiona todas las operaciones relacionadas con la creación, recuperación y eliminación de chats, mientras que el router de autenticación implementa los mecanismos de registro y acceso de usuarios.

Esta organización implica que cada módulo puede definirse de forma autónoma. Un router incluye

- la instancia de APIRouter,
- las rutas HTTP,
- las dependencias necesarias para cada *endpoint*
- y los esquemas de entrada y salida utilizados para la serialización de datos.

Un fragmento representativo de la definición de un router es el siguiente:

```
router = APIRouter()

@router.get(
    CONVERSATIONS,
    response_model=chat_schema.ConversationSummaries,
)
async def get_user_chat_list(
    user: User = Depends(current_user),
    db: AsyncSession = Depends(get_db),
):
    ...
```

En este ejemplo, el router define un endpoint que recupera el listado de conversaciones del usuario autenticado. Las dependencias necesarias para ejecutar la operación, como la sesión asíncrona de base de datos o el usuario autenticado, se inyectan automáticamente mediante el sistema de dependencias de FastAPI. De esta forma, el router puede centrarse únicamente en la lógica de la operación que expone.

El registro de los routers en la aplicación principal se realiza de forma explícita. Cada módulo se incorpora al servicio mediante una llamada a `include_router`, lo que permite definir un prefijo común y etiquetas de clasificación utilizadas por la documentación automática de la API. Esta inicialización se realiza en el módulo principal, donde se importan los routers definidos en los distintos paquetes de la API. La aplicación actúa únicamente como punto de ensamblado, delegando la implementación de cada conjunto de *endpoints* en su módulo correspondiente. Este patrón evita concentrar la lógica del servicio en un único archivo y favorece la organización del código en unidades coherentes.

La modularización también facilita la reutilización de dependencias comunes. Elementos como la autenticación, la obtención de la sesión asíncrona o el acceso al cliente Redis se definen como dependencias independientes que pueden utilizarse en distintos routers sin duplicar código. Este enfoque reduce el acoplamiento entre módulos y permite modificar la implementación de estas dependencias sin alterar los routers que las utilizan.

Además de la organización por dominios funcionales, la API incorpora una separación adicional para gestionar distintas versiones del servicio. Los routers correspondientes a la versión 2 se encuentran en el subpaquete `api.v2`. Esta estructura permite mantener simultáneamente distintas variantes de un mismo endpoint, facilitando la transición entre versiones sin interrumpir el funcionamiento de clientes existentes.

La separación entre versiones se implementa replicando la organización modular utilizada en la API principal. Cada router de la versión 2 define sus propias rutas y esquemas, adaptados a las convenciones de diseño adoptadas. La coexistencia de ambas versiones permite introducir cambios en el en el diseño de los endpoints manteniendo al mismo tiempo compatibilidad con integraciones previas.

En conjunto, el enfoque modular adoptado en la API responde a tres objetivos principales: mantener una separación clara de responsabilidades, facilitar la evolución del sistema y mejorar la mantenibilidad del código. La utilización de routers independientes, combinada con el sistema de dependencias de FastAPI, permite estructurar el servicio como un conjunto de componentes desacoplados que se integran en la aplicación principal únicamente en el momento de inicialización.

Sistema de autenticación

El acceso a los recursos de la API está protegido mediante un sistema de autenticación basado en JSON Web Tokens (JWT). Este mecanismo permite verificar la identidad de los usuarios y autorizar el acceso a los distintos endpoints sin necesidad de mantener estado de sesión en el servidor. El diseño sigue un modelo de autenticación **stateless**, donde la información necesaria para validar una solicitud se transmite en cada petición mediante un token firmado criptográficamente.

La implementación del sistema de autenticación se basa en la biblioteca **fastapi-users**, que proporciona una capa de integración entre FastAPI y los mecanismos habituales de autenticación en aplicaciones web. Esta biblioteca abstrae parte de la lógica necesaria para gestionar usuarios, generar tokens y validar credenciales, permitiendo concentrar la implementación en la configuración del flujo de autenticación y en la integración con los endpoints de la API.

El sistema se organiza en torno a un conjunto de routers especializados que exponen las operaciones necesarias para la gestión de identidades. Estas rutas incluyen los procesos de registro, autenticación y verificación de usuarios. Cada uno de estos procesos se implementa mediante endpoints HTTP que interactúan con el gestor de usuarios proporcionado por la biblioteca.

Entre los endpoints disponibles se encuentran las rutas de autenticación, que permiten a un cliente obtener un token JWT tras proporcionar credenciales válidas. Este proceso constituye el punto de entrada al sistema para los usuarios registrados.

Durante la fase de autenticación, el servidor valida las credenciales del usuario utilizando el gestor de usuarios configurado en el sistema. Si las credenciales son correctas, se genera un token JWT firmado con la clave secreta configurada en el servidor. Este token contiene información mínima sobre la identidad del usuario y su estado de autenticación.

El cliente debe almacenar el token recibido y enviarlo en cada solicitud posterior a los endpoints protegidos de la API. Para ello se utiliza la cabecera HTTP estándar Authorization, que transporta el token siguiendo el esquema de autenticación Bearer: "Authorization: Bearer <token>".

Cuando el servidor recibe una solicitud que incluye esta cabecera, el middleware de autenticación valida el token. Este proceso consiste en verificar la firma criptográfica del JWT y comprobar que el token no ha expirado. Si la validación es correcta, el sistema recupera la identidad del usuario asociado al token y la inyecta en el endpoint correspondiente mediante el sistema de dependencias de FastAPI.

El acceso a los recursos protegidos se implementa mediante dependencias que garantizan que la solicitud proviene de un usuario autenticado. Estas dependencias se definen a partir de los mecanismos proporcionados por **fastapi-users**. El patrón habitual consiste en declarar una dependencia que recupera el usuario actual y exigir su resolución antes de ejecutar el endpoint.

Un ejemplo representativo es el siguiente:

```
current_user = fastapi_users.current_user(active=True)

router = APIRouter(
    dependencies=[Depends(current_user)]
)
```

En esta configuración, cualquier endpoint definido dentro del router requiere que la solicitud esté asociada a un usuario autenticado y activo. Si la validación del token falla o el usuario no existe, la solicitud se rechaza automáticamente antes de que el endpoint sea ejecutado.

Este mecanismo permite centralizar el control de acceso sin necesidad de repetir comprobaciones en cada endpoint. La autenticación se resuelve como parte del proceso de resolución de dependencias del framework, lo que simplifica la lógica de los controladores y mantiene una separación clara entre la verificación de identidad y la lógica funcional de la API.

Además del control de acceso, el sistema de autenticación permite recuperar la identidad del usuario dentro de los endpoints. Esto resulta necesario para asociar las operaciones realizadas con el usuario que las ejecuta. Por ejemplo, los endpoints relacionados con la gestión de conversaciones utilizan el identificador del usuario autenticado para recuperar únicamente los recursos que le pertenecen.

Un ejemplo de uso dentro de un endpoint es el siguiente:

```
async def get_user_chat_list(
    user: User = Depends(current_user),
    db: AsyncSession = Depends(get_db),
):
```

En este caso, el parámetro `user` contiene la información del usuario autenticado, resuelta automáticamente a partir del token JWT incluido en la petición. Esto permite restringir las operaciones a los recursos asociados a ese usuario sin necesidad de recibir explícitamente su identificador en la solicitud.

El sistema de autenticación también se integra con la documentación automática generada por FastAPI. La especificación OpenAPI describe los esquemas de autenticación utilizados por la API, lo que permite a Swagger UI solicitar credenciales al usuario y generar automáticamente las cabeceras de autorización necesarias para probar los endpoints protegidos.

Desde el punto de vista de la arquitectura del servicio, el sistema de autenticación constituye una capa transversal que afecta a la mayoría de los routers de la API. Su integración mediante dependencias permite aplicar políticas de acceso de forma uniforme sin introducir dependencias directas entre los distintos módulos del sistema.

La elección de un mecanismo basado en JWT responde a varios criterios operativos. En primer lugar, elimina la necesidad de mantener sesiones persistentes en el servidor, lo que simplifica el despliegue del servicio en entornos distribuidos. Cada instancia del backend puede validar los tokens de forma independiente sin consultar un almacén centralizado de sesiones. En segundo lugar, el modelo stateless reduce la carga en el servidor y facilita el escalado horizontal del servicio.

Sesiones asíncronas

La API utiliza **SQLAlchemy en modo asíncrono** para realizar las operaciones de acceso a datos que requieren los distintos endpoints. Para ello se emplea el tipo de sesión `AsyncSession`, diseñado para integrarse con aplicaciones que utilizan programación asíncrona sobre un servidor ASGI, como es el caso de FastAPI ejecutándose sobre Uvicorn.

El uso de sesiones asíncronas responde al modelo de ejecución del framework. FastAPI se basa en corutinas y en un bucle de eventos que permite gestionar múltiples solicitudes concurrentes. En este contexto, las operaciones que implican entrada o salida, como las consultas a la base de datos, deben ejecutarse de forma no bloqueante para evitar que una petición impida el avance de otras tareas en el servidor.

`AsyncSession` permite realizar estas operaciones mediante llamadas asíncronas (`await`). Cuando una operación de acceso a datos está en curso, el control se devuelve temporalmente al bucle de eventos, lo que permite que el servidor continúe procesando otras solicitudes mientras se completa la operación. Este comportamiento mejora la capacidad del sistema para gestionar múltiples clientes concurrentes y reduce el riesgo de bloqueos en el servidor.

Dentro de la API, la sesión asíncrona se proporciona a los endpoints mediante una dependencia denominada `get_db`. Esta función encapsula el proceso de creación de

la sesión y se integra con el sistema de dependencias de FastAPI. Los endpoints que necesitan interactuar con la capa de persistencia simplemente declaran esta dependencia como parámetro de la función que implementa el endpoint.

El ciclo de vida de la sesión está asociado a la duración de la solicitud HTTP. Cada petición que requiere acceso a datos recibe su propia instancia de `AsyncSession`. Una vez finalizado el procesamiento de la solicitud, la sesión se cierra automáticamente, liberando los recursos asociados. Este modelo evita que distintas solicitudes compartan el mismo contexto de sesión, lo que contribuye a prevenir interferencias entre operaciones concurrentes.

Este mecanismo permite desacoplar la lógica de los endpoints de los detalles de configuración de la sesión. Los routers se limitan a utilizar la sesión proporcionada, mientras que la creación, configuración y cierre de la misma se gestionan de forma centralizada en la capa de dependencias.

Redis

La API integra **Redis** como componente auxiliar de infraestructura para gestionar determinadas operaciones que requieren coordinación entre solicitudes o entre distintas instancias del servicio. Redis se utiliza como un almacén de datos en memoria accesible mediante red, lo que permite realizar operaciones de lectura y escritura con una latencia reducida.

El acceso a Redis se realiza mediante un cliente que se obtiene a través de la dependencia `get_redis`. Esta dependencia encapsula la configuración de la conexión con el servidor Redis y proporciona al endpoint una instancia lista para utilizar. De este modo, los routers pueden emplear Redis sin necesidad de conocer los detalles de inicialización del cliente.

La integración en un endpoint sigue el mismo patrón utilizado para otras dependencias.

El uso principal de Redis dentro de la API está relacionado con la gestión de concurrencia en determinadas operaciones. Algunas acciones del sistema, como el procesamiento de una conversación o la interacción con componentes externos, requieren garantizar que un mismo recurso no sea procesado simultáneamente por múltiples solicitudes.

En un entorno donde el servidor puede recibir múltiples peticiones concurrentes, y donde incluso pueden existir varias instancias del backend ejecutándose en paralelo, resulta necesario disponer de un mecanismo que permita coordinar estas operaciones.

Redis se utiliza para implementar este mecanismo mediante estructuras de datos simples que actúan como bloqueos temporales.

El funcionamiento conceptual de este mecanismo puede describirse de la siguiente forma. Cuando un endpoint inicia una operación que requiere exclusividad, intenta registrar un bloqueo asociado al recurso que se va a procesar. Si el bloqueo no existe, el sistema lo crea y permite continuar con la operación. Mientras el bloqueo esté activo, otras solicitudes que intenten procesar el mismo recurso no podrán iniciar la operación correspondiente.

El uso de Redis para este tipo de coordinación resulta adecuado porque proporciona operaciones atómicas y tiempos de respuesta reducidos. Además, al tratarse de un servicio accesible por red, puede ser utilizado por todas las instancias del backend, lo que permite mantener la coherencia del sistema incluso en arquitecturas distribuidas.

Desde el punto de vista de la API, Redis se utiliza como un componente de soporte que interviene únicamente en aquellos endpoints donde es necesario controlar la concurrencia. Su integración mediante el sistema de dependencias permite que los routers accedan al cliente Redis de forma uniforme, manteniendo la separación entre la lógica de los endpoints y la configuración de los componentes de infraestructura.

Middlewares y mecanismos de seguridad de la API

La API incorpora varios mecanismos de seguridad implementados principalmente mediante middlewares HTTP y configuraciones de infraestructura del framework. Estos mecanismos se aplican de forma transversal a todas las solicitudes que recibe el servicio y tienen como objetivo reforzar la protección frente a ataques habituales en aplicaciones web, controlar el acceso al sistema y garantizar un comportamiento predecible ante escenarios de uso intensivo o abusivo.

En FastAPI, un middleware es un componente que intercepta cada solicitud HTTP antes de que este alcance el endpoint correspondiente, y también puede intervenir en la generación de la respuesta. Este modelo permite introducir lógica común a todas las rutas del servicio sin necesidad de duplicar código en los distintos routers. En la API, los middlewares se utilizan para aplicar políticas de seguridad, configurar el acceso desde distintos orígenes y limitar el volumen de solicitudes que puede realizar un cliente.

El middleware se implementa como una función asíncrona registrada mediante el decorador `@app.middleware("http")`. Su estructura general consiste en recibir la solicitud entrante, delegar el procesamiento en el siguiente componente de la cadena y, una vez obtenida la respuesta, añadir las cabeceras de seguridad antes de devolverla al cliente.

Entre las cabeceras configuradas se encuentran las siguientes:

- **Strict-Transport-Security** (HSTS)
Obliga a los navegadores a realizar futuras solicitudes únicamente mediante HTTPS. Esta cabecera reduce el riesgo de ataques de tipo downgrade o interceptación de tráfico mediante HTTP.
- **Content-Security-Policy** (CSP)
Define una política que controla qué tipos de recursos pueden cargarse en el contexto del navegador. Esta política ayuda a prevenir ataques de **Cross-Site Scripting (XSS)** al limitar el origen de scripts, estilos u otros contenidos ejecutables.
- **X-Content-Type-Options**
Impide que el navegador intente deducir el tipo de contenido de una respuesta cuando este no coincide con el encabezado declarado. Esta restricción reduce el riesgo de ataques basados en interpretación incorrecta de tipos MIME.
- **X-Frame-Options**
Evita que la aplicación pueda ser incrustada dentro de un iframe en dominios externos. Esta medida protege frente a ataques de **clickjacking**, en los que una interfaz se superpone de forma engañosa sobre la aplicación.
- **X-XSS-Protection**
Activa mecanismos de protección contra ataques de scripting presentes en algunos navegadores.
- **Referrer-Policy**
Controla la información que el navegador envía en la cabecera Referer al navegar entre páginas. Esto evita la exposición innecesaria de información sobre rutas internas del sistema.
- **Permissions-Policy**
Limita el acceso a ciertas funcionalidades del navegador, como sensores o APIs de hardware, cuando el contenido se ejecuta en un contexto web.

Estas cabeceras se añaden automáticamente a todas las respuestas generadas por la API. De esta forma, cualquier cliente que acceda al servicio recibe una configuración de seguridad coherente sin necesidad de que los endpoints individuales intervengan en este proceso.

Cabe señalar que la política CSP se ajusta de forma específica para permitir el funcionamiento de la documentación interactiva generada por FastAPI (Swagger UI y ReDoc). Estas interfaces requieren cargar scripts dinámicos, por lo que la política se configura para permitir su ejecución únicamente en los endpoints destinados a la documentación

Otro componente relevante es el middleware de **Cross-Origin Resource Sharing (CORS)**. Este mecanismo regula las solicitudes HTTP que provienen de dominios distintos al del servidor que expone la API.

En aplicaciones modernas, especialmente aquellas que incluyen interfaces web separadas del backend, es habitual que el frontend se ejecute en un dominio distinto. Sin un mecanismo de control adecuado, los navegadores bloquearían estas solicitudes por razones de seguridad. El middleware CORS permite definir explícitamente qué orígenes pueden acceder a la API y en qué condiciones.

La configuración del middleware establece los siguientes parámetros:

- lista de orígenes permitidos
- métodos HTTP aceptados
- cabeceras que pueden enviarse en las solicitudes
- posibilidad de enviar credenciales en la comunicación

El middleware se registra durante la inicialización de la aplicación mediante `app.add_middleware`.

La lista de orígenes permitidos se define en la configuración del sistema, lo que permite adaptar el comportamiento del servicio a distintos entornos de despliegue, como desarrollo, pruebas o producción.

Además de las medidas anteriores, la API incorpora un mecanismo de limitación de tasa de solicitudes destinado a prevenir el uso abusivo del servicio. Este mecanismo controla el número máximo de peticiones que puede realizar un cliente dentro de un intervalo de tiempo determinado.

La implementación se basa en la biblioteca `slowapi`, que se integra con `FastAPI` para aplicar restricciones de forma automática sobre las rutas del servicio. El sistema identifica a los clientes utilizando la dirección IP de origen de la solicitud y aplica los límites configurados.

Despliegue

La API está diseñada para ejecutarse como una aplicación ASGI utilizando el servidor `Uvicorn`, que proporciona un entorno de ejecución optimizado para aplicaciones Python basadas en programación asíncrona. Este tipo de servidor es especialmente adecuado para frameworks como `FastAPI`, ya que permite aprovechar el modelo de ejecución basado en corutinas y gestionar múltiples conexiones concurrentes con un consumo reducido de recursos.

El punto de entrada de la aplicación se encuentra en el módulo principal que define la instancia de FastAPI. El servidor Uvicorn se encarga de cargar esta aplicación y gestionar las conexiones HTTP entrantes. Durante la ejecución, cada solicitud recibida por el servidor se procesa mediante el bucle de eventos asíncrono que coordina la ejecución de las corutinas definidas en los endpoints.

Para facilitar el despliegue del servicio en distintos entornos, la aplicación se encapsula dentro de una imagen Docker. El uso de contenedores permite empaquetar la aplicación junto con sus dependencias y configuraciones necesarias, garantizando que el servicio se ejecute de forma consistente independientemente del entorno donde se despliegue.

La imagen base utilizada para construir el contenedor se basa en una distribución ligera de Python. Sobre esta imagen se instalan las dependencias del proyecto y se copia el código fuente de la aplicación. Posteriormente se define un script de arranque que inicializa el servidor Uvicorn y configura los parámetros necesarios para la ejecución del servicio.

El uso de Docker facilita además la integración del backend con sistemas de orquestación o plataformas de despliegue automatizado. En entornos de producción, el contenedor puede ejecutarse como parte de una infraestructura compuesta por varios servicios, lo que permite escalar la API horizontalmente cuando aumenta la carga de solicitudes.

Durante el arranque del contenedor se ejecuta un script que actúa como punto de entrada del sistema. Este script se encarga de preparar el entorno de ejecución y lanzar el servidor ASGI. En algunos casos, también puede configurar parámetros adicionales relacionados con el uso de certificados TLS o con la integración con otros servicios de la infraestructura.

Una vez iniciado el servidor, la aplicación queda accesible a través del puerto configurado. Los clientes pueden interactuar con la API mediante solicitudes HTTP enviadas a las rutas correspondientes. La documentación interactiva generada por FastAPI también se publica en este entorno, lo que permite explorar los endpoints disponibles y realizar pruebas de forma directa.

El despliegue basado en contenedores permite reproducir el mismo entorno tanto en desarrollo como en producción. Esto reduce las diferencias entre entornos y facilita la detección de errores durante el ciclo de desarrollo. Además, el uso de un servidor ASGI optimizado y de un framework diseñado para programación asíncrona contribuye a mantener un rendimiento adecuado incluso cuando el sistema debe gestionar múltiples solicitudes simultáneas.

Frontend

Para conectarse a la API se han desarrollado varios clientes: Inicialmente se creó una interfaz por la línea de comandos sencilla para pruebas en producción y más adelante dos interfaces web alternativas.

API_CLI

La aplicación API_CLI constituye el cliente de línea de comandos diseñado para interactuar con la API REST del sistema. Su objetivo es proporcionar una interfaz textual que permita a un usuario ejecutar las operaciones principales del sistema sin necesidad de utilizar herramientas externas como navegadores o clientes HTTP especializados. La CLI actúa como una capa de cliente que encapsula la comunicación HTTP con la API y proporciona un flujo de interacción estructurado a través del terminal.

La implementación de la CLI se basa en el uso de Python y en bibliotecas estándar para el procesamiento de argumentos y la comunicación HTTP. La interacción con la API se realiza principalmente mediante la biblioteca `httpx`, que permite realizar solicitudes HTTP de forma eficiente.

La arquitectura de la aplicación sigue un modelo modular que separa claramente tres responsabilidades principales: control de la interacción con el usuario, comunicación con la API y utilidades de interfaz de consola. Esta separación facilita el mantenimiento del código y permite que cada componente evolucione de forma independiente.

Los módulos principales que componen la CLI son los siguientes: `api_cli`, `main.py`, `console_cli.py`, `Authentication.py`, `Chat.py` y `Menu.py`.

El módulo `main.py` actúa como punto de entrada de la aplicación. Su responsabilidad consiste en procesar los argumentos de línea de comandos, validar la configuración de conexión con la API y lanzar el controlador principal de la interfaz. Este diseño permite que el usuario configure parámetros como la dirección del servidor API en el momento de iniciar la aplicación.

Entre las responsabilidades de este módulo se encuentran: Procesar argumentos mediante `argparse`, validar la dirección base de la API, inicializar el controlador principal e iniciar el ciclo de interacción con el usuario.

El módulo `console_cli.py` implementa el controlador principal de la aplicación. Este componente coordina la interacción entre el usuario y los distintos servicios que proporciona la API. El controlador gestiona el ciclo de ejecución de la interfaz, muestra los menús disponibles y delega las operaciones concretas en los módulos especializados.

Las responsabilidades principales del controlador incluyen: Mostrar el menú principal, gestionar la autenticación del usuario, iniciar conversaciones, enviar mensajes y recuperar historiales de chat.

```
-----
Diligencia CLI
-----

Select an option:
1. Login
2. Register
3. Show AI models
4. Show templates

0. Cancel / Back

> |
```

Figura 7: Menú inicial de la CLI.

Las operaciones relacionadas con la autenticación se encapsulan en el módulo `authentication.py`. Este componente implementa las funciones necesarias para registrar usuarios, iniciar sesión, cerrar sesión y eliminar cuentas. Internamente, estas operaciones se traducen en solicitudes HTTP dirigidas a los endpoints de autenticación expuestos por la API.

```
-----
Diligencia CLI
-----

Enter email: prueba@prueba.es
Enter password: PruebaPrueba@1

[!] Login successful.

|
```

Figura 8: Inicio de sesión.

La gestión de conversaciones se implementa en el módulo `chat.py`. Este módulo actúa como un cliente especializado para los endpoints de chat, proporcionando funciones que permiten crear conversaciones, enviar mensajes y recuperar el historial asociado a una conversación. De esta forma, la lógica de comunicación con los endpoints de chat queda separada de la lógica de interfaz de usuario.

```
-----
Diligencia CLI
Logged as: prueba@prueba.es
-----
1. AI: ¿Qué es el phishing?
2. AI: **¿Qué es el phishing?**

El *phishing* (del inglés *phishing* = "pescar") es una técnica de ingeniería social que los ciberdelincuentes usan para **engañarte y conseguir información confidencial** (como contraseñas, datos bancarios, números de tarjeta, etc.). Lo hacen haciéndose pasar por una entidad de confianza (banco, empresa, administración, amigos, etc.) y enviándote mensajes que parecen legítimos.

### ¿Cómo funciona?
1. **Mensaje atractivo**
   - Un correo electrónico, SMS, mensaje en redes sociales o incluso una llamada telefónica.
   - Suele contener un asunto urgente ("¡Tu cuenta ha sido bloqueada!", "Necesitamos confirmar tu pago", "Has ganado un premio!").
```

Figura 9: Ejemplo de conversación. El renderizado de Markdown no está soportado.

Por último, el módulo menu.py proporciona las utilidades necesarias para la interacción con el terminal. Este módulo se encarga de mostrar menús, leer entradas del usuario, limpiar la pantalla y gestionar el formato de salida. De esta manera, el resto de la aplicación puede centrarse en la lógica funcional sin preocuparse por los detalles de presentación.

```
Logged as: prueba@prueba.es
-----

Select an option:
1. New chat
2. Select chat
3. Show chats
4. Delete chat
5. Log out
6. Delete user
7. Show AI models
8. Show templates

0. Cancel / Back

> |
```

Figura 10: Menú de la aplicación cliente una vez iniciada la sesión.

```
-----
Diligencia CLI
Logged as: prueba@prueba.es
-----

Name      Description
-----
1. gpt-oss:120b
2. gemma3:27b
3. qwen3:32b

Press Enter to return...|
```

Figura 11: Listado de modelos de IA disponibles.

```
-----  
Diligencia CLI  
Logged as: prueba@prueba.es  
-----  
  
Select an option:  
  1. Preguntas de phishing [Asistente base]  
  
  0. Cancel / Back  
  
> |
```

Figura 12: Listado de conversaciones creadas por el usuario. Entre corchetes se muestra la plantilla utilizada.

Alternativa web propia

La aplicación sigue un modelo de cliente web desacoplado, donde toda la lógica de negocio reside en la API y el frontend se limita a gestionar la interacción con el usuario y la comunicación HTTP.

La estructura se organiza en torno a componentes funcionales que encapsulan las distintas áreas de la aplicación, manteniendo una separación clara entre presentación y acceso a datos.

El cliente emplea un módulo centralizado para la comunicación con la API. Este módulo construye y envía las solicitudes HTTP necesarias, gestionando tanto los parámetros como las cabeceras.

El patrón seguido consiste en definir funciones específicas para cada operación relevante, que encapsulan: La URL del *endpoint*, el método HTTP, los datos de entrada y el tratamiento de la respuesta.

Las solicitudes autenticadas incluyen el token JWT en la cabecera. Este token se obtiene tras el proceso de autenticación y se reutiliza durante toda la sesión del usuario.

Para obtenerlo, el cliente proporciona una interfaz para el registro y autenticación de usuarios. Tras introducir las credenciales, estas se envían al *backend* y, en caso de éxito, se recibe un token JWT.

Este token se almacena en el estado de la aplicación y se utiliza para:

- Mantener la sesión activa,
- habilitar el acceso a funcionalidades protegidas
- y autenticar las solicitudes posteriores.



The image shows a login interface for an application named 'Digiliencia'. At the top, there is a header with a red square icon containing a white 'D' and the text 'Digiliencia' in bold, followed by 'INCIBE · Asistente IA'. Below this, there are two tabs: 'Iniciar sesión' (highlighted with a red underline) and 'Registrarse'. The main form area contains two input fields: 'Correo electrónico' with the placeholder 'usuario@incibe.es' and 'Contraseña' with a masked password represented by dots. At the bottom of the form is a large red button labeled 'Iniciar sesión'.

Figura 13: Cuadro de inicio de sesión.

El componente principal de la aplicación es la interfaz de chat, que permite interactuar con el sistema de conversaciones expuesto por la API.

Las funcionalidades implementadas incluyen la creación de nuevas conversaciones, la selección de conversaciones existentes, la visualización del historial de mensajes y el envío de nuevos mensajes.

Cada acción del usuario se traduce en una solicitud a la API, cuya respuesta se integra en la interfaz.

El estado de la conversación activa se mantiene en el cliente, lo que permite actualizar dinámicamente la vista sin recargar la página. El cliente mantiene un estado interno que incluye:

- usuario autenticado
- token de sesión
- conversación activa
- historial de mensajes

Cada vez que se recibe una respuesta de la API, el estado se actualiza y la interfaz se vuelve a renderizar para reflejar los nuevos datos.

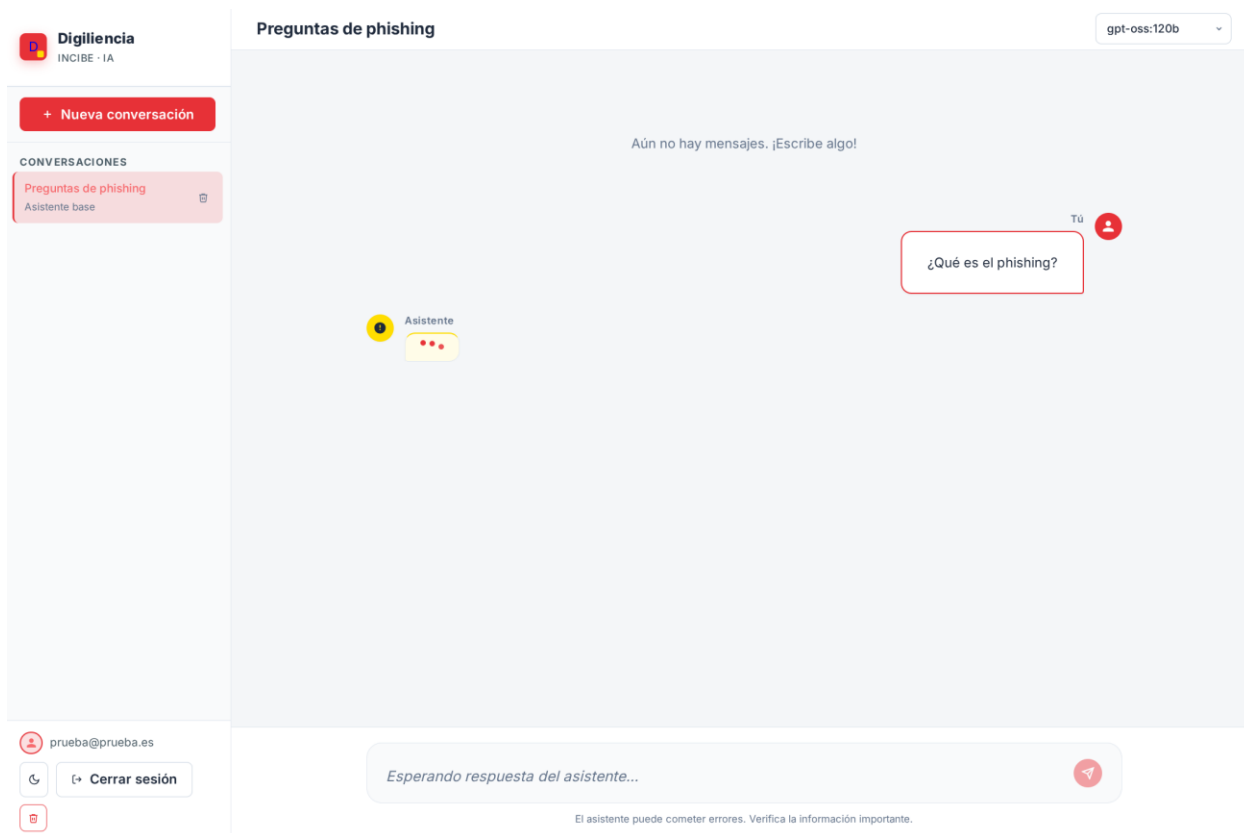


Figura 14: Interfaz de la aplicación web.

Digitilancia
INCIBE - IA

+ Nueva conversación

CONVERSACIONES

Preguntas de phishing
Asistente base

prueba@prueba.es

Cerrar sesión

Preguntas de phishing

gpt-oss:120b

pueden ser más vulnerables.

¿Qué hacer si sospechas que has recibido un intento de phishing?

1. **No interactúes** con el mensaje (no hagas clic, no descargues archivos, no respondas).
2. **Marca el correo como spam o elimínalo.**
3. **Informa a la entidad** a través de su canal oficial (teléfono, web, app).
4. **Cambia la contraseña** de la cuenta afectada si ya introdujiste datos, y revisa la actividad reciente.
5. **Contacta con tu banco** o la entidad financiera si crees que tus datos bancarios pueden estar comprometidos.

Resumen rápido

- **Phishing** = técnica de engaño para robar datos.
 - Busca **urgencia, errores, enlaces sospechosos y solicitudes de datos.**
 - **Protección:** verifica remitentes, no hagas clic directo, usa 2FA, mantén todo actualizado y reporta los intentos.
- ¡Con estos hábitos podrás reconocer y evitar la mayoría de los intentos de phishing! Si tienes algún caso concreto que quieras analizar, cuéntamelo y te ayudo a revisarlo.

Tú

Explícame brevemente qué es el phishing en menos de dos párrafos

Asistente

El *phishing* es una técnica de ingeniería social en la que los ciberdelincuentes se hacen pasar por una entidad de confianza (banco, empresa, administración, etc.) mediante correos, SMS o mensajes en redes sociales, con el objetivo de engañarte para que reveles datos confidenciales como contraseñas, números de tarjeta o información personal. Normalmente incluyen un mensaje urgente o atractivo y un enlace o archivo que lleva a una página falsa muy parecida a la original, donde se solicitan esos datos.

Escribe tu mensaje... (Enter para enviar, Shift+Enter para nueva línea)

El asistente puede cometer errores. Verifica la información importante.

Figura 15: Ejemplo de conversación.

El cliente también permite consultar los modelos de LLM disponibles y las plantillas definidas en el sistema, aunque estas últimas no se han llegado a implementar. Esta información se obtiene mediante llamadas a los endpoints correspondientes de la API.

La interfaz presenta estos elementos de forma estructurada, permitiendo al usuario seleccionar configuraciones antes de iniciar una interacción.

gpt-oss:120b

gpt-oss:120b

gemma3:27b

qwen3:32b

Figura 16: Selección del modelo de IA empleado. La selección se muestra en la esquina superior derecha.

Finalmente, el cliente interpreta los códigos de estado HTTP devueltos por la API y adapta su comportamiento en consecuencia. En caso de error, se muestra información al usuario que permite identificar el problema.

Se contemplan, entre otros, los siguientes escenarios:

- credenciales incorrectas
- acceso no autorizado
- errores en la solicitud
- fallos del servidor

El tratamiento de errores se centraliza en el módulo de comunicación con la API, lo que garantiza un comportamiento uniforme en toda la aplicación.

The image shows a login interface for 'Digiliencia' by INCIBE. At the top, there's a header with the 'D' logo and the text 'Digiliencia INCIBE · Asistente IA'. Below this are two tabs: 'Iniciar sesión' (selected) and 'Registrarse'. The form contains two input fields: 'Correo electrónico' with the value 'prueba@prueba.es' and 'Contraseña' with masked characters. A red 'Iniciar sesión' button is at the bottom of the form. At the very bottom of the screen, a dark red error banner displays the message: 'No se pudo conectar al servidor. Verifica la conexión.' with close buttons on either side.

Figura 17: Ejemplo de error causado por una pérdida de conexión con el servidor.

Alternativa web en Wordpress

De forma alternativa se ha colaborado con un equipo de la universidad de Salamanca para crear una nueva interfaz web basada en Wordpress. Esta página se encuentra alojada en la dirección digiliencia.org.

La página web permite realizar consultas sin iniciar sesión. No obstante, ello implica que no se conservan las conversaciones entre sesiones.

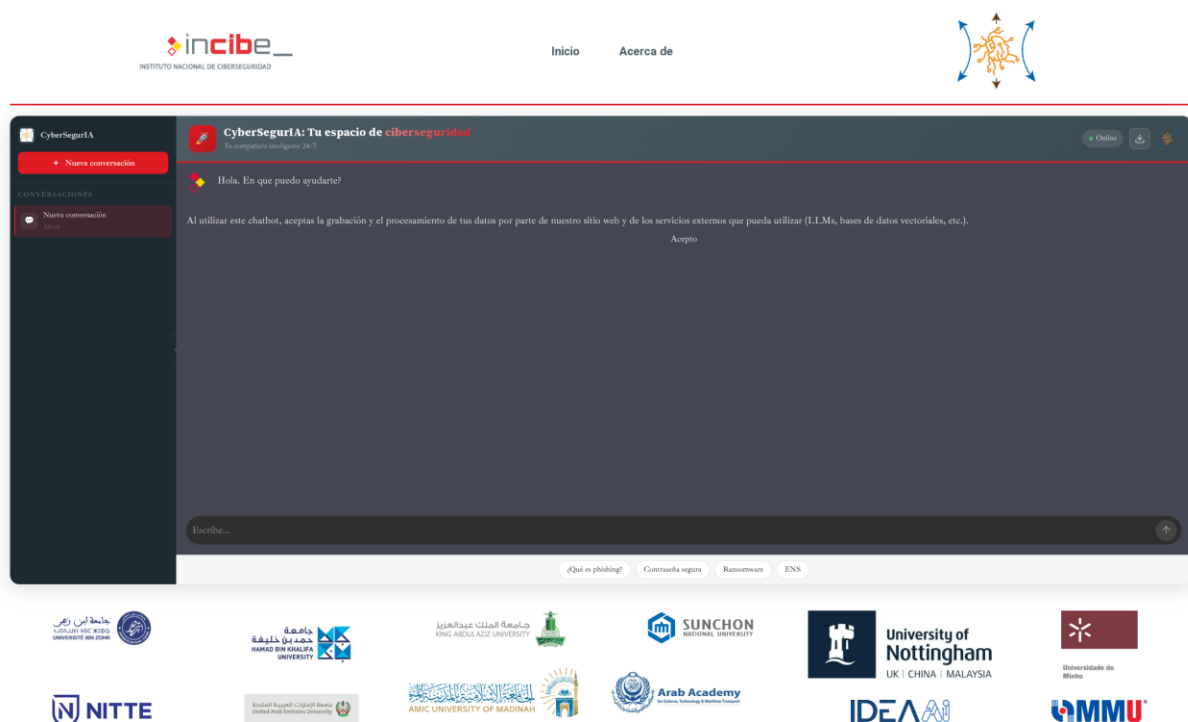


Figura 18: Interfaz web alternativa.

Despliegue en SCAYLE

El despliegue en el Centro de Supercomputación de Castilla y León (SCAYLE) se ha configurado en dos zonas de disponibilidad lógica, interconectadas mediante una red virtual privada (VLAN) de 10 Gbps para garantizar una latencia mínima en la transferencia de tensores y grafos.

- Zona de Servicios: Esta zona aloja tres máquinas virtuales.
 - VM-01 (Persistencia de Grafos): Nodo dedicado a la base de datos Neo4j. Configuración de memoria Heap ajustada al 50% de la RAM disponible y Page Cache al 40% para maximizar el rendimiento de las consultas Cypher.

- VM-02 (Persistencia Relacional y Cache): Alojamiento de PostgreSQL y Redis. Gestiona la integridad referencial de usuarios, sesiones y colas de tareas.
- VM-03 (Orquestación y API): Servidor de aplicaciones Python (FastAPI) que implementa una interfaz de tipo REST. Actúa como puerta de enlace para la lógica de negocio y transformación de datos. También incluye un Reverse Proxy (Nginx) que actúa como terminador TLS.
- Zona de Computación:
 - Nodo Físico de Aceleración: Servidor físico equipado con una GPU NVIDIA A100 (80GB VRAM). Este nodo opera de forma exclusiva los contenedores de inferencia de Inteligencia Artificial.

Resultados obtenidos

Análisis de problemas y gestión de riesgos

Durante la ejecución del proyecto, el equipo técnico se enfrentó a desafíos significativos que impactaron en el cronograma y obligaron a replanificar hitos. A continuación, se detallan las incidencias y las medidas de mitigación adoptadas.

Retraso en la Infraestructura (SCAYLE)

- **Incidencia:** La disponibilidad de los recursos de supercomputación en el centro SCAYLE sufrió una demora administrativa y técnica de seis meses. Esto impidió realizar pruebas de carga real y entrenamiento de modelos durante la primera mitad del proyecto.
- **Impacto:** Imposibilidad de validar la inferencia de modelos grandes (70B parámetros) en las fases iniciales.
- **Mitigación:** Se optó por una estrategia de desarrollo en "entornos sombra". El equipo utilizó estaciones de trabajo locales con GPUs de gama consumo (NVIDIA RTX) y modelos cuantizados de menor tamaño (7B parámetros) para desarrollar la lógica. Una vez disponible el acceso al SCAYLE, se realizó una migración masiva del contenedor Docker, validando la portabilidad de la arquitectura diseñada.

Obsolescencia de los Scrapers (Caso Web Forum)

- **Incidencia:** Las medidas de seguridad anti-scraping del World Economic Forum y cambios en el DOM de otras fuentes provocaron que los scripts de ingestión fallaran recurrentemente, interrumpiendo el flujo de datos.
- **Mitigación:** Se estableció un protocolo de mantenimiento correctivo semanal. Se migró de Requests a Selenium (navegación completa) para simular comportamiento humano, asumiendo un mayor coste computacional a cambio de garantizar la adquisición del dato.

Cambios en el Equipo de Desarrollo

- **Incidencia:** La baja voluntaria del ingeniero Esteban Peña Salazar en la fase media del proyecto supuso la pérdida de conocimiento específico sobre la configuración inicial de los pipelines de CI/CD y los scripts de limpieza de datos.

Conclusiones

El proyecto MARC.0 (Digiliencia) finaliza con la entrega de una plataforma de inteligencia en ciberseguridad diseñada para ingerir información no estructurada y

servirla a través de un modelo conversacional. El sistema cubre el ciclo de vida del dato desde su extracción hasta su uso en inferencia de lenguaje natural.

El módulo de *scraping* desarrollado en Python ha permitido generar una base de conocimiento con más de 30.000 artículos y noticias sobre ciberseguridad.

Las fuentes de información empleadas son bastante heterogéneas: se han empleado bases de datos de vulnerabilidades técnicas; guías de buenas prácticas y avisos, como información dirigida a usuarios no expertos para mejorar su postura de seguridad; y noticias y datos de actualidad, para el procesamiento de eventos en tiempo real con el objetivo de poder alertar sobre campañas de fraude o nuevas amenazas detectadas por agencias nacionales e internacionales.

En materia de seguridad, la plataforma cumple con los requisitos de soberanía del dato y privacidad clave del proyecto. Esto es importante puesto que el servicio maneja información potencialmente sensible sobre incidentes y consultas de los usuarios.

Además, el sistema RAG garantiza la trazabilidad de las respuestas, mejora su predictibilidad y reduce la posibilidad de alucinación, alineándose con las directivas europeas en materia de inteligencia artificial.

Lineas de trabajo futuras

El sistema desarrollado permite integrarse y aportar valor a servicios de soporte en ciberseguridad, como la línea de ayuda 017. Sirviendo como apoyo al personal encargado de la asistencia a los ciudadanos.

A nivel operativo, la integración del asistente conversacional permitiría responder de forma automática a consultas recurrentes y problemas comunes de los ciudadanos, aplicando las metodologías y taxonomías de INCIBE. Esto facilitaría un filtrado previo de las incidencias, permitiendo derivar a los agentes humanos (niveles N1, N2 y N3) únicamente los casos que requieran atención especializada. De forma complementaria, también puede ser empleado por los agentes humano para agilizar y mejorar el servicio ofrecido asistiéndoles en la búsqueda de información, por ejemplo.

La arquitectura RAG también dota al proyecto de gran flexibilidad para ser adaptado a otros cometidos proporcionando distintos dataversos o complementándolo. También es posible emplear información confidencial mediante sistemas de control de acceso que limiten la información a la que el agente tiene acceso en función del usuario que realiza la consulta.

Anexo

Especificación de la API REST

Este documento define la interfaz de programación de aplicaciones (API) RESTful para la comunicación entre el cliente web y el servidor de lógica de negocio. Convenciones Generales:

Formato de Datos: Todas las peticiones y respuestas que contengan un cuerpo de datos utilizarán el formato application/json.

Autenticación: Los endpoints que requieren autenticación deben recibir un Bearer Token en la cabecera HTTP Authorization. o Authorization: Bearer <token_de_sesion>

Parámetros de Ruta: Los parámetros dinámicos en las rutas se indican con llaves, ej: /api/users/{userId}.

Gestión de Errores: Se utilizan códigos de estado HTTP estándar (200, 201, 400, 401, 404, 500) para indicar el resultado de las operaciones.

Autenticación (Auth)

Iniciar Sesión (JSON)

- **Método:** POST
- **Ruta:** /api/auth/login
- **Acción:** Autentica a un usuario mediante email y contraseña en formato JSON y devuelve un token JWT de acceso.
- **Autenticación requerida:** No
- **Cuerpo (Body):** UserLogin
 - email (string, EmailStr): Dirección de correo electrónico registrada.
 - password (string): Contraseña del usuario.
- **Respuesta:**
 - 200 OK: Login exitoso. Devuelve { "access_token": "...", "token_type": "bearer" }.
 - 400 Bad Request: Formato de credenciales no válido.
 - 401 Unauthorized: Email o contraseña incorrectos, o usuario inactivo.

Iniciar Sesión (OAuth2 Form / JWT Estándar)

- **Método:** POST

- **Ruta:** /api/auth/jwt/login
- **Acción:** Autentica a un usuario mediante formulario OAuth2 (form-data con username y password) y devuelve un token JWT. Este es el endpoint estándar de fastapi-users.
- **Autenticación requerida:** No
- **Cuerpo (Body):** Form data (OAuth2PasswordRequestForm)
 - username (string): Email del usuario.
 - password (string): Contraseña del usuario.
- **Respuesta:**
 - 200 OK: Login exitoso. Devuelve { "access_token": "...", "token_type": "bearer" }.
 - 400 Bad Request: Credenciales no válidas.

Cerrar Sesión (JWT Estándar)

- **Método:** POST
- **Ruta:** /api/auth/jwt/logout
- **Acción:** Cierra la sesión del usuario autenticado. En la autenticación JWT stateless, este endpoint es informativo; el cliente debe descartar el token localmente.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
 - 200 OK: Logout exitoso.
 - 401 Unauthorized: Usuario no autenticado.

Registrar Usuario

- **Método:** POST
- **Ruta:** /api/auth/register
- **Acción:** Crea una nueva cuenta de usuario en el sistema.
- **Autenticación requerida:** No
- **Cuerpo (Body):** UserRegistration
 - email (string, EmailStr): Dirección de correo electrónico para el registro.
 - password (string, min. 8 caracteres): Contraseña del usuario. Debe contener al menos una letra y un número.
- **Respuesta:**
 - 201 Created: Usuario registrado con éxito. Devuelve UserRead (id, email, is_active, is_superuser, is_verified).
 - 400 Bad Request: El email ya existe o la contraseña no cumple los requisitos de seguridad.

Solicitar Verificación de Cuenta

- **Método:** POST
- **Ruta:** /api/auth/request-verify-token
- **Acción:** Envía un token de verificación al email del usuario para validar su cuenta.
- **Autenticación requerida:** No
- **Cuerpo (Body):**
 - email (string, EmailStr): Email del usuario a verificar.
- **Respuesta:**
 - 202 Accepted: Solicitud de verificación procesada.
 - 400 Bad Request: El usuario ya está verificado o no existe.

Verificar Cuenta

- **Método:** POST
- **Ruta:** /api/auth/verify
- **Acción:** Verifica la cuenta de un usuario mediante el token recibido por email.
- **Autenticación requerida:** No
- **Cuerpo (Body):**
 - token (string): Token de verificación recibido por email.
- **Respuesta:**
 - 200 OK: Cuenta verificada con éxito. Devuelve UserRead.
 - 400 Bad Request: Token no válido, expirado o usuario ya verificado.

Gestión de Usuarios (Users)

Obtener Perfil del Usuario Actual

- **Método:** GET
- **Ruta:** /api/users/me
- **Acción:** Recupera la información del perfil del usuario autenticado.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
 - 200 OK: Perfil recuperado con éxito. Devuelve UserRead (id, email, is_active, is_superuser, is_verified).
 - 401 Unauthorized: Usuario no autenticado.

Actualizar Perfil del Usuario Actual

- **Método:** PATCH
- **Ruta:** /api/users/me
- **Acción:** Actualiza la información del perfil del usuario autenticado. Los campos sensibles (contraseña, is_superuser) son ignorados por seguridad.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** UserUpdate (todos los campos son opcionales)
- email (string, EmailStr, opcional): Nuevo email.
- password (string, opcional): Nueva contraseña.
- **Respuesta:**
- 200 OK: Perfil actualizado con éxito. Devuelve UserRead.
- 400 Bad Request: El email ya está en uso por otra cuenta o datos no válidos.
- 401 Unauthorized: Usuario no autenticado.

Eliminar Cuenta del Usuario Actual

- **Método:** DELETE
- **Ruta:** /api/users/me
- **Acción:** Elimina permanentemente la cuenta del usuario autenticado.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 204 No Content: Cuenta eliminada con éxito.
- 401 Unauthorized: Usuario no autenticado.

Cerrar Sesión de Usuario

- **Método:** POST
- **Ruta:** /api/users/me
- **Acción:** Cierra la sesión del usuario autenticado. En autenticación JWT stateless, confirma al cliente que debe descartar el token.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: Logout exitoso. Devuelve { "detail": "Logout successful" }.
- 401 Unauthorized: Usuario no autenticado.

Conversaciones / Chats

Listar Conversaciones del Usuario

- **Método:** GET
- **Ruta:** /api/chats/conversations
- **Acción:** Recupera un listado con los resúmenes de todas las conversaciones del usuario autenticado.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: Lista de conversaciones recuperada con éxito. Devuelve ConversationSummaries con un array de ConversationSummary (id, title, ai_prompt_id).
- 401 Unauthorized: Usuario no autenticado.

Obtener Conversación Completa

- **Método:** GET
- **Ruta:** /api/chats/{chat_id}
- **Acción:** Recupera los detalles completos de una conversación específica, incluyendo todo el historial de mensajes.
- **Autenticación requerida:** Sí (Bearer Token)
- **Parámetros de ruta:**
- chat_id (UUID): Identificador único de la conversación.
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: Conversación recuperada con éxito. Devuelve ConversationFull (id, title, ai_prompt_id, messages[]).
- 401 Unauthorized: Usuario no autenticado.
- 404 Not Found: Conversación no encontrada o no pertenece al usuario.

Enviar Mensaje a Conversación

- **Método:** PATCH
- **Ruta:** /api/chats/{chat_id}
- **Acción:** Procesa un mensaje del usuario y genera una respuesta de la IA. Utiliza un sistema de bloqueo con Redis para prevenir solicitudes concurrentes sobre la misma conversación.
- **Autenticación requerida:** Sí (Bearer Token)
- **Parámetros de ruta:**
- chat_id (UUID): Identificador único de la conversación.
- **Cuerpo (Body):** MessageInput

- **content** (string, min. 1 carácter): Texto del mensaje del usuario.
- **model_id** (UUID, opcional): UUID del modelo de IA a utilizar. Si no se especifica, se utiliza el modelo por defecto.
- **Respuesta:**
- 200 OK: Mensaje procesado con éxito. Devuelve AIResponse con el campo text (respuesta generada por la IA).
- 401 Unauthorized: Usuario no autenticado.
- 404 Not Found: Conversación no encontrada, no pertenece al usuario, o modelo no encontrado.
- 409 Conflict: Ya hay una solicitud en proceso para esta conversación (chat ocupado).
- 500 Internal Server Error: Error al generar la respuesta de la IA.

Crear Nueva Conversación

- **Método:** PATCH
- **Ruta:** /api/chats
- **Acción:** Crea una nueva conversación de chat asociada al usuario autenticado.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** ChatCreate
- **title** (string, 1–255 caracteres): Título descriptivo de la conversación.
- **ai_prompt_id** (UUID, opcional): UUID de la plantilla/prompt de IA a utilizar. Si no se especifica, se asigna la plantilla por defecto.
- **Respuesta:**
- 201 Created: Conversación creada con éxito. Devuelve ConversationSummary (id, title, ai_prompt_id).
- 400 Bad Request: Plantilla de IA no válida o no encontrada.
- 401 Unauthorized: Usuario no autenticado.

Importar Conversación

- **Método:** PUT
- **Ruta:** /api/chats
- **Acción:** Crea una nueva conversación importando en bloque una lista de mensajes existentes.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** ConversationImport
- **title** (string, 1–255 caracteres): Título de la conversación a importar.
- **ai_prompt_id** (UUID, opcional): UUID de la plantilla/prompt de IA. Si no se especifica, se usa la plantilla por defecto.

- **texts** (array de AIResponse): Lista ordenada de mensajes a importar, cada uno con un campo text (string).
- **Respuesta:**
- 201 Created: Conversación importada con éxito.
- Devuelve ConversationSummary (id, title, ai_prompt_id).
- 400 Bad Request: El UUID de ai_prompt_id proporcionado no existe.
- 401 Unauthorized: Usuario no autenticado.
- 500 Internal Server Error: No existe una plantilla por defecto configurada.

Eliminar Conversación

- **Método:** DELETE
- **Ruta:** /api/chats/{chat_id}
- **Acción:** Elimina permanentemente una conversación y todos sus mensajes asociados. También limpia la sesión del agente de IA.
- **Autenticación requerida:** Sí (Bearer Token)
- **Parámetros de ruta:**
- chat_id (UUID): Identificador único de la conversación a eliminar.
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 204 No Content: Conversación eliminada con éxito.
- 401 Unauthorized: Usuario no autenticado.
- 404 Not Found: Conversación no encontrada o no pertenece al usuario.

Plantillas de IA (Templates)

Listar Plantillas Disponibles

- **Método:** GET
- **Ruta:** /api/chats/templates
- **Acción:** Recupera un listado de todas las plantillas de prompt de IA configuradas en el sistema.
- **Autenticación requerida:** No
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: Lista de plantillas recuperada con éxito. Devuelve Templates con un array de TemplateSummary (id, name, description).

Obtener Plantilla por ID

- **Método:** GET
- **Ruta:** /api/chats/templates/{template_id}
- **Acción:** Recupera la información de una plantilla de prompt de IA específica por su ID.
- **Autenticación requerida:** No
- **Parámetros de ruta:**
- template_id (UUID): Identificador único de la plantilla.
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: Plantilla recuperada con éxito. Devuelve TemplateSummary (id, name, description).
- 404 Not Found: Plantilla no encontrada.

Modelos de IA (Models)

Listar Modelos Disponibles

- **Método:** GET
- **Ruta:** /api/chats/models
- **Acción:** Recupera un listado de todos los modelos de IA disponibles y configurados en el sistema.
- **Autenticación requerida:** No
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: Lista de modelos recuperada con éxito. Devuelve Models con un array de ModelSummary (id, name).

Obtener Modelo por ID

- **Método:** GET
- **Ruta:** /api/chats/models/{model_id}
- **Acción:** Recupera la información de un modelo de IA específico por su ID.
- **Autenticación requerida:** No
- **Parámetros de ruta:**
- model_id (UUID): Identificador único del modelo.
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: Modelo recuperado con éxito. Devuelve ModelSummary (id, name).
- 404 Not Found: Modelo no encontrado.

Sistema (System)

Raíz de la API

- **Método:** GET
- **Ruta:** /api/
- **Acción:** Devuelve un mensaje de bienvenida confirmando que la API está operativa.
- **Autenticación requerida:** No
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: { "message": "Welcome to the Digiliencia Chat API v2" }.

Health Check

- **Método:** GET
- **Ruta:** /api/health
- **Acción:** Verifica que el servicio FastAPI está en funcionamiento.
- **Autenticación requerida:** No
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: { "status": "ok", "service": "fastapi-service" }.

API V2 (Endpoints Equivalentes)

La API V2 expone exactamente la misma funcionalidad que V1 bajo el prefijo /api/v2/. A continuación, se listan las rutas equivalentes:

Autenticación V2

Iniciar Sesión (JSON)

- **Método:** POST
- **Ruta:** /api/v2/auth/login
- **Acción:** Equivalente a V1. Autentica a un usuario mediante email y contraseña en formato JSON y devuelve un token JWT.
- **Autenticación requerida:** No
- **Cuerpo (Body):** UserLogin (email, password).
- **Respuesta:**
- 200 OK: Login exitoso. Devuelve { "access_token": "...", "token_type": "bearer" }.
- 400 Bad Request: Credenciales no válidas.
- 401 Unauthorized: Email o contraseña incorrectos.

Iniciar Sesión (OAuth2 Form / JWT Estándar)

- **Método:** POST
- **Ruta:** /api/v2/auth/jwt/login
- **Acción:** Equivalente a V1. Autenticación mediante formulario OAuth2.
- **Autenticación requerida:** No
- **Cuerpo (Body):** Form data (username, password).
- **Respuesta:**
 - 200 OK: Login exitoso.
 - 400 Bad Request: Credenciales no válidas.

Cerrar Sesión (JWT Estándar)

- **Método:** POST
- **Ruta:** /api/v2/auth/jwt/logout
- **Acción:** Equivalente a V1. Cierra sesión del usuario autenticado.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
 - 200 OK: Logout exitoso.
 - 401 Unauthorized: Usuario no autenticado.

Registrar Usuario

- **Método:** POST
- **Ruta:** /api/v2/auth/register
- **Acción:** Equivalente a V1. Crea una nueva cuenta de usuario.
- **Autenticación requerida:** No
- **Cuerpo (Body):** UserRegistration (email, password).
- **Respuesta:**
 - 201 Created: Usuario registrado con éxito. Devuelve UserRead.
 - 400 Bad Request: Email ya existe o contraseña no válida.

Solicitar Verificación de Cuenta

- **Método:** POST
- **Ruta:** /api/v2/auth/request-verify-token
- **Acción:** Equivalente a V1. Envía token de verificación al email del usuario.
- **Autenticación requerida:** No

- **Cuerpo (Body):** { "email": "..." }.
- **Respuesta:**
- 202 Accepted: Solicitud procesada.
- 400 Bad Request: Usuario ya verificado o no existe.

Verificar Cuenta

- **Método:** POST
- **Ruta:** /api/v2/auth/verify
- **Acción:** Equivalente a V1. Verifica la cuenta mediante token.
- **Autenticación requerida:** No
- **Cuerpo (Body):** { "token": "..." }.
- **Respuesta:**
- 200 OK: Cuenta verificada. Devuelve UserRead.
- 400 Bad Request: Token no válido o expirado.

Gestión de Usuarios V2

Obtener Perfil del Usuario Actual

- **Método:** GET
- **Ruta:** /api/v2/users/me
- **Acción:** Equivalente a V1. Recupera el perfil del usuario autenticado.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: Devuelve UserRead.
- 401 Unauthorized: Usuario no autenticado.

Actualizar Perfil del Usuario Actual

- **Método:** PATCH
- **Ruta:** /api/v2/users/me
- **Acción:** Equivalente a V1. Actualiza el perfil del usuario autenticado.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** UserUpdate (email, password — todos opcionales).
- **Respuesta:**
- 200 OK: Perfil actualizado. Devuelve UserRead.
- 400 Bad Request: Email ya en uso.
- 401 Unauthorized: Usuario no autenticado.

Eliminar Cuenta del Usuario Actual

- **Método:** DELETE
- **Ruta:** /api/v2/users/me
- **Acción:** Equivalente a V1. Elimina permanentemente la cuenta del usuario.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
 - 204 No Content: Cuenta eliminada.
 - 401 Unauthorized: Usuario no autenticado.

Cerrar Sesión de Usuario

- **Método:** POST
- **Ruta:** /api/v2/users/me
- **Acción:** Equivalente a V1. Cierra sesión del usuario.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
 - 200 OK: { "detail": "Logout successful" }.
 - 401 Unauthorized: Usuario no autenticado.

Conversaciones / Chats V2

Listar Conversaciones del Usuario

- **Método:** GET
- **Ruta:** /api/v2/chats/conversations
- **Acción:** Equivalente a V1. Lista los resúmenes de las conversaciones del usuario.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
 - 200 OK: Devuelve ConversationSummaries.
 - 401 Unauthorized: Usuario no autenticado.

Obtener Conversación Completa

- **Método:** GET
- **Ruta:** /api/v2/chats/{chat_id}

- **Acción:** Equivalente a V1. Recupera la conversación completa con historial de mensajes.
- **Autenticación requerida:** Sí (Bearer Token)
- **Parámetros de ruta:**
- chat_id (UUID): Identificador de la conversación.
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
- 200 OK: Devuelve ConversationFull.
- 401 Unauthorized: Usuario no autenticado.
- 404 Not Found: Conversación no encontrada.

Enviar Mensaje a Conversación

- **Método:** PATCH
- **Ruta:** /api/v2/chats/{chat_id}
- **Acción:** Equivalente a V1. Procesa un mensaje del usuario y genera la respuesta de la IA.
- **Autenticación requerida:** Sí (Bearer Token)
- **Parámetros de ruta:**
- chat_id (UUID): Identificador de la conversación.
- **Cuerpo (Body):** MessageInput (content, model_id opcional).
- **Respuesta:**
- 200 OK: Devuelve AIResponse.
- 401 Unauthorized: Usuario no autenticado.
- 404 Not Found: Conversación o modelo no encontrado.
- 409 Conflict: Chat ocupado.
- 500 Internal Server Error: Error al generar respuesta.

Crear Nueva Conversación

- **Método:** PATCH
- **Ruta:** /api/v2/chats
- **Acción:** Equivalente a V1. Crea una nueva conversación de chat.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** ChatCreate (title, ai_prompt_id opcional).
- **Respuesta:**
- 201 Created: Devuelve ConversationSummary.
- 400 Bad Request: Plantilla de IA no válida.
- 401 Unauthorized: Usuario no autenticado.

Importar Conversación

- **Método:** PUT
- **Ruta:** /api/v2/chats
- **Acción:** Equivalente a V1. Importa una conversación en bloque.
- **Autenticación requerida:** Sí (Bearer Token)
- **Cuerpo (Body):** ConversationImport (title, ai_prompt_id opcional, texts[]).
- **Respuesta:**
 - 201 Created: Devuelve ConversationSummary.
 - 400 Bad Request: UUID de plantilla no existe.
 - 401 Unauthorized: Usuario no autenticado.

Eliminar Conversación

- **Método:** DELETE
- **Ruta:** /api/v2/chats/{chat_id}
- **Acción:** Equivalente a V1. Elimina una conversación y sus mensajes.
- **Autenticación requerida:** Sí (Bearer Token)
- **Parámetros de ruta:**
 - chat_id (UUID): Identificador de la conversación.
- **Cuerpo (Body):** Ninguno.
- **Respuesta:**
 - 204 No Content: Conversación eliminada.
 - 401 Unauthorized: Usuario no autenticado.
 - 404 Not Found: Conversación no encontrada.

Participantes

Han tomado parte en el proyecto:

- Ander Chueca Rodríguez
- Sergio González Rebollo
- Esteban Peña Salazar
- Álvaro Prieto Álvarez
- Carlos Prieto Viñuela
- Santiago Ares Alfayate
- César Bermejo Fernández

y como directores:

- Enrique López González
- Cristina Mendaña Cuervo